

CMPE-260
Digital System Design 2
Laboratory Manual

Department of Computer Engineering
Rochester Institute of Technology

Spring 2255

Contents

Timeline	iii
Late Policy	v
Submission Guidelines	vii
0 Reintroduction to VHDL (Optional)	1
1 Introduction to Vivado & Simple ALU	7
2 Register File	17
3 Instruction Fetch and Decode Stages	27
4 Execute Stage	41
5 Memory & Writeback Stage	55
6 Pipelined MIPS Processor	67
7 Processor Timing (Extra Credit)	75
A Xilinx Vivado Tutorial	79
B notN Source Code	87
C sllN Source Code	89
D globals Package (library)	91
E alu4 Source Code	93
F aluTB Source Code	95
G Xilinx Design Constraints File	97
H Clock Wizard Tutorial	99
I EDAPlayground Tutorial	101
J ModelSim Tutorial	103
K Common Vivado Problems	105

Timeline

NOTE: timeline is subject to change at professor discretion.

Week	Lab	Name
1	Lab 1	Introduction to Vivado & Simple ALU
2	Lab 2	Register File
3	Lab 2	Register File
4	Lab 2	Register File
5	Lab 3	Instruction Fetch & Decode
6	Lab 3	Instruction Fetch & Decode
7	Lab 4	Execute Stage
8	Lab 4	Execute Stage
9	Spring Break	
10	Lab 5	Memory & Write Back
11	Lab 6	Complete MIPS
12	Lab 6	Complete MIPS
13	No Lab	
14	Lab 7	MIPS Timing
15	Lab 7	MIPS Timing

Late Policy

- All lab reports are due one week after the exercise is completed, by midnight.
- If a report is not submitted by the initial due date, it carries the following penalties:
 - Within 2 days late: -10%
 - Within 2 - 14 days late: -30%
 - Within 14 - 21 days late: -60%
 - More than 21 days late: -100%
- Lab submissions must be to the MyCourses assignment and must contain all relevant VHDL designs, testbenches, and constraints file [see Submission Guidelines for further details].
- All labs are weighted equally.
- In the event of extenuating circumstances, speak with the lab instructor or TAs. Speak up as early as possible. The later issues are brought to the lab instructor/ TAs, it harder it is for them to help.

Submission Guidelines

These instructions apply to all submissions in the course

1. Everything must be in a PDF, except code. Code must be submitted as a .txt file.
2. You must include *all* of the required items for full credit, make it look professional.
3. Everything must be typed and proofread. *NOTHING HANDWRITTEN WILL BE ACCEPTED FOR POINTS.*
4. Submit all code to MyCourses as .txt files.
5. Submit grading sheets to MyCourses appended to the back of the report or worksheet for that exercise (scan it in). They must be signed by the lab instructor or a TA to get any credit on the exercise.
6. Submit the worksheets and reports to MyCourses.
7. All entries not submitted by the beginning of the next lab are considered late, unless otherwise specified.
8. *DO NOT ZIP YOUR FILES.*
9. If you have any questions, email *ALL* TAs for your section.

Exercise 0 Reintroduction to VHDL (Optional)

Introduction

Welcome to Digital System Design II! The lab portion of this course focuses on the design of the microprocessor without interlocked pipelined stages (MIPS) processor and its stages, and the efficiency of each stage and its pipeline will be focused on in CMPE-350, Computer Organization.

In the lab portion of this course, students will learn how to design, test, and verify hardware description language (HDL) designs through various means of simulation and design synthesis. Students will also learn how to implement and test their designs on a field programmable gate array (FPGA), namely the Digilent Basys3. Throughout the exercises in this course, students will design, test, and verify each “stage” of a MIPS processor. By the last few exercise(s), students will have a working MIPS processor that can be programmed via the Instruction Memory. The design will then execute the given instructions and output the results.

It is imperative that students have a good basic knowledge of VHSIC Hardware Description Language (VHDL), which was taught in CMPE-160, Digital System Design I. If students are unsure about or have forgotten VHDL, it is highly recommended to do the exercise in Lab 0. Please reach out to your TAs or instructor(s) if you have any questions.

Objective

In this exercise, students will design and create a four-bit four-to-one multiplexer using both structural and behavioral design architectures. For a structural multiplexer, this will require making each individual logic gate component and connecting them into an overarching multiplexer design. Students will use tools provided by VHDL to describe the behavior of the multiplexer in a behavioral design. Both designs will be simulated and verified using a student-designed self-checking testbench that contains predefined test cases held in a test record array.

Hardware Specifications

The design must meet the following specifications and use all components listed at least once. For this exercise, the four-bit 4:1 Multiplexer will be implemented both structurally and behaviorally.

- **3 input AND gate**

- 1-bit input "A"
- 1-bit input "B"
- 1-bit input "C"
- 1-bit output "Y"

- **4 input OR gate**

- 1-bit input "A"
- 1-bit input "B"
- 1-bit input "C"
- 1-bit input "D"
- 1-bit output "Y"

- **NOT gate**

- 1-bit input "A"
- 1-bit output "Y"

- **Four Bit 4-to-1 Multiplexer**

- 4-bit input "A"
- 4-bit input "B"
- 4-bit input "C"
- 4-bit input "D"
- 2 bit input "SEL"
- 4-bit output "Y"

4:1 Multiplexer Block Diagram

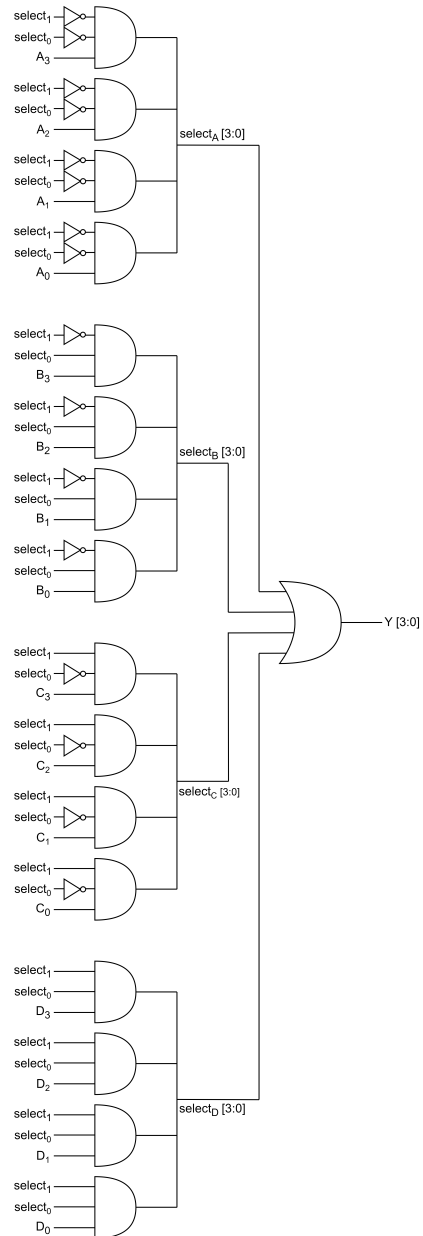


Figure 0.1: 4:1 Multiplexer Block Diagram

A structural 4:1 mux needs to implement the following function:

$$\text{MUX}_{4:1} = \overline{S_1} \overline{S_0} A + \overline{S_1} S_0 B + S_1 \overline{S_0} C + S_1 S_0 D$$



Where A, B, C, and D are 4-bit vectors:

$$A = A_3 \& A_2 \& A_1 \& A_0$$

$$B = B_3 \& B_2 \& B_1 \& B_0$$

$$C = C_3 \& C_2 \& C_1 \& C_0$$

$$D = D_3 \& D_2 \& D_1 \& D_0$$

Remember that in Boolean algebra, S_1 and S_0 are going to be distributed among the bits of A, B, C and D.

Procedure

1. Create a new project in Vivado and name it "Lab0".
2. Create new VHDL files for the behavioral designed AND, OR and NOT gates, respectively.



Refer to Example.vhd for a review on VHDL programming.

3. Using the Hardware Descriptions, implement the logic gates in each file.



Refer to the Hardware Specifications for the I/O ports of the components.

4. Create a new VHDL file for the structural design of a 4:1 multiplexer.
5. Implement the structural designed 4:1 multiplexer in the new VHDL file using port maps based on Figure 0.1.



Use the behavioral designed AND OR and NOT gates as components in the structural design for the mux.

6. Create a new VHDL file for the behavioral design of a 4:1 multiplexer.
7. Implement the behavioral designed 4:1 multiplexer in the new VHDL file based on Figure 0.1.



For the behavioral mux, implement the logic directly without using a port map.

8. Create a testbench for the 4:1 multiplexer. **Test both the structural and behavioral design, they should have the same output.**



Refer to Example_tb.vhd for a review on Testbenches

Testbench Tutorial

Write a testbench that will test your behavioral and structural 4:1 multiplexer. The two designs should produce the same output where:

- If $SEL = 00$, then $Y = A$
- If $SEL = 01$, then $Y = B$
- If $SEL = 10$, then $Y = C$
- If $SEL = 11$, then $Y = D$

Where $A = 1010$, $B = 1011$, $C = 1100$, $D = 1101$.

A review on how to create a testbench is available in Example_tb.vhd.

No Grading Sheet for this lab, as it is not mandatory.

Exercise 1 Introduction to Vivado & Simple ALU

Introduction

This exercise will detail the basic functionality of the Xilinx Vivado Design Suite. This software suite provides an integrated development environment (IDE) for the design, synthesis, and implementation of custom designs for reconfigurable hardware. Students will learn how to create a custom design from source and simulate, synthesize, implement, and load it onto a Basys3 FPGA. Students will first complete a tutorial for a 4-bit ALU supporting five operations. The design will be expanded to support additional operations and a 32-bit width in a future exercise.

Pre-lab Activities

- Print the sign-off sheet.

Procedure

Part 1

1. Follow the Xilinx Vivado Tutorial in Appendix A to start the design for a partial ALU.



When copying VHDL files from the Xilinx Vivado Tutorial, do NOT copy and paste from the PDF! Hidden characters will get copied and mess up the VHDL files in Vivado. Also note that (.xdc) files are CASE SENSITIVE.

Hardware Demonstration

Follow the Hardware tutorial in Appendix A to implement the partial ALU design on the Basys3 FPGA. The ALU should be able to perform the Invert & (Shift Left Logical) SLL operations. Demonstrate two different cases of each operation to the TA for credit.

Part 2

1. To create the more comprehensive partial ALU, create a component for each of the operations in Table 1.1 and assign them to the corresponding opcode in the top-level ALU design. Make sure that each component is declared with generic bit width and is instantiated in the ALU with a width of 32 bits. Ensure that the following 4-bit opcodes are used. While only six operations will be supported now, this will allow many more operations to be added in later exercises.

Table 1.1: Operations

Opcode	Operation
1000	Logical OR
1010	Logical AND
1011	Logical XOR
1100	Shift Left Logical (SLL)
1101	Shift Right Logical (SRL)
1110	Shift Right Arithmetic (SRA)

- Write a testbench for the ALU, similar to the one provided in the tutorial. This testbench should be self-checking and use a record array to contain the necessary test cases. It should also have user-friendly assert statements for debugging purposes. It should spot-check **at least** four “generic” cases for **each** operation as well as the following “edge” cases:

Value1	Operation	Value2
0x6	SRL	0x2
0x6	SRA	0x1
0x6	SRA	0x2
0xF0000000	SRA	0x1
0x0	OR	0x0
0x0	OR	0xF
0xF	OR	0xF
0x5	OR	0xA
0xA	OR	0x5
0x0	XOR	0x0
0x0	XOR	0xF
0xF	XOR	0x0
0xF	XOR	0xF
0x5	XOR	0xA
0xA	XOR	0x5
0x0	AND	0x0
0x0	AND	0xF
0xF	AND	0x0
0xF	AND	0xF



The proper syntax for hexadecimal in VHDL is: `x"F"`

- Using the testbench, perform both a Behavioral Simulation and a Post-Implementation Timing Simulation. Check the results for accuracy.
- Create a block diagram of the full ALU for the lab report. NOTE: A block diagram is not a schematic; it is a series of labeled shapes that illustrate a design. Any digital drawing software will work, previously success was seen with Draw.IO, lucid charts, and GIMP. Hand drawn schematics, no matter how nice, will be a zero.



A block diagram is different than a schematic. A schematic shows the internals of each component, with all logic components included. A block diagram consists of “black boxes” with inputs and outputs that make up the highest level of the internals of the design.

- Write a full lab report describing the ALU created, specifically citing how it was designed and its functionality was verified. See this exercise’s lab report deliverables for more details.



In the lab report, be sure to cite specific examples of proof that the waveforms are correct. Saying that the wave matched the expected results with no test cases cited does not explain what the expected results were, or how they matched the testbench.

A good test vector array

A good test record array will generally follow the format: Actual Input, vs Expected output. The details in the actual input is what will go inside, and stimulate your design, whereas the expected output should be compared against your actual output (the signal you declared above from your top level component), using an assert. e.g. In this your top level design will include, A,B,OP,Y. So your test record array should look something like this:

```
type testRecord is record
a, b, y : std_logic_vector(N-1 downto 0);
op : std_logic_vector(3 downto 0);
end record;

-- Array of all tests.
-- Format: ( A, B, Y, OP )

type testArray is array(natural range <>) of testRecord;

constant testArr : testArray := (
-- NOT tests
(x"0000000f", x"00000000", x"00000000", x"0"),
(x"ffff0000", x"00000005", x"000000f0", x"1"),
(x"00000001", x"00000000", x"00000009", x"2"),
(x"00000002", x"00000000", x"00000008", x"3"),
-- Add more test cases as needed
);
```

Then later on in your test bench you should have a process that looks like:

```
op = tester(i).op;
a <= testArr(i).a;
b <= testArr(i).b;
wait for 100ns;

-- Wait and assert expected and actual results
assert testArr(i).y = y
    report "Test failed"
    severity failure;
```

Exercise 1: Introduction to Vivado & Simple ALU

Student's Name: _____

Section: _____

Demo		Point Value	Points Earned	Date
Part 1: 4-bit ALU	Behavioral Simulation	4		
	Post-Synthesis Timing Simulation	4		
	Synthesis Schematic	4		
	Synthesis Utilization Report	4		
	Post-Implementation Timing Simulation	4		
	RTL Schematic for srlN shifter	4		
	Hardware Demonstration	4		
Part 2: 32-bit ALU	Behavioral Simulation	16		
	Post-Implementation Timing Simulation	16		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

Exercise 1: Introduction to Vivado & Simple ALU

Report		Point Value	Points Earned	Comments
Abstract		5		
Design Methodology	Full ALU Block Diagram	5		
	Description of SRL and SRA Design	5		
Results & Analysis	Parts 1 and 2 Behavioral Waveforms	5		
	Parts 1 and 2 Post-Implementation Timing Waveforms	5		
	Explanation of Test Cases	5		
Conclusion		5		
Source Code (Style)		5		
Total for prelab, demo, and report		100		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

In-Lab Deliverables

Part 1: 4-bit ALU

- VHDL Code for generic, 4-bit, two operation ALU.
 - VHDL Code for generic, 4-bit NOT.
 - VHDL Code for generic, 4-bit Logical Left Shift.
- VHDL Testbench Code for generic, 4-bit, two operation ALU.
- VHDL Testbench must successfully perform behavioral, post-synthesis timing, and post-implementation timing simulations.
- Synthesis schematic and utilization report for 4-bit ALU.
- VHDL Code for generic, 4-bit Logical Right Shift.
- RTL schematic for 4-bit Logical Right shift.
- Hardware demonstration of 4-bit, two operation ALU on Basys3 FPGA.

Part 2: 32-bit ALU

- VHDL Code for generic, 32-bit, five operation ALU. (Modified from Part 1)
 - VHDL Code for generic, 32-bit Logical OR.
 - VHDL Code for generic, 32-bit Logical AND.
 - VHDL Code for generic, 32-bit Logical XOR.
 - VHDL Code for generic, 32-bit Logical Left Shift.
 - VHDL Code for generic, 32-bit Logical Right Shift.
 - VHDL Code for generic, 32-bit Arithmetic Right Shift.
- VHDL Testbench Code for generic, 32-bit, five operation ALU.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must have four generic test cases for each 32-bit ALU operation.
 - Testbench must contain all “edge” cases highlighted in step 3 of the procedure section of this exercise.
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL Testbench must successfully perform behavioral and post-implementation timing simulations.

Lab Report Deliverables

Cover Page

- Proper cover page that includes exercise number and name as well as student's name, lab section, lab instructor, teaching assistants, and lecture instructor.

Abstract

- Objective of the exercise.
- Summary of what was done.
- How the design was tested.
- Results of exercise (successful or unsuccessful).

Design Methodology

- Block Diagram of Full 32-bit ALU. Note that a block diagram is different than a schematic. A schematic shows the internals of each component, with all logic components included. A block diagram consists of “black boxes” with inputs and outputs that make up the highest level of the internals of the design.
- Description of shift right logical and shift right arithmetic design.
- Discussion of generic N-bit design.

Results and Analysis

- Waveforms and discussion of behavioral and post-implementation timing simulations. Cite specific examples of proof that the waveforms are correct. Stating that the wave matched the expected results with no test cases cited does not explain what the expected results were, or how they matched the testbench.
- Discussion of test cases.

Conclusion

- Restate objective and summary of exercise.
- State the final results of the exercise.
- Reflections of lessons learned and discussion of future use of what was learned.

Other

- Report should be original and follow digital system design report guidelines.

Other Deliverables

- All VHDL code must be properly commented and have proper comment headers that detail the student's name, lab section, and file description.
- All submissions must follow the submission guidelines detailed within the preliminary matter of this manual.
- **ALL VHDL CODE FOR BOTH PARTS 1 AND 2 MUST BE SUBMITTED TO MYCOURSES AS .TXT FILES TO RECEIVE A GRADE.**

Exercise 2 Register File

Objective

This exercise investigates the design of a register file for a FPGA. The objective of this exercise is to implement a digital system that is capable of storing multiple words, or groups of bits, as well as understanding how to properly test such a design.

Background

Computers need to be able to store and access information very quickly. However, simple digital systems like D flip flops, by themselves, cannot hold enough information to be useful. Thus register files are created that allow for one bus to access multiple locations, each which can hold multiple bits. In this exercise, there are two outputs, which allows for two addresses to be read from at the same time, increasing the speed at which the necessary data can be obtained.

Pre-lab Activities

- Read Chapter 6 section 2 of **Digital Design and Computer Architecture, 2nd Edition** by Harris and Harris and answer the following questions.
 - What is assembly language?
 - In the assembly code *sub a,b,c*, which operand is the destination?
 - Why are operands stored in registers?
 - This lab creates 8 registers. How many registers does a MIPS have?
 - How many operands can be read from a MIPS register at a time?
 - How many operands can be written to a MIPS register at a time?
 - In your own words describe the distinction between machine language & assembly language.
 - how many bits are typically 1 register.
- Review the use of generics in both entity declarations and instantiations.
- Complete the given testbench shell from the MyCourses.
 - Add 7 additional test cases.
 - Complete assert statement.
- Print the sign-off sheet

Hardware Specification

The design must meet the following specifications and use all components listed at least once. Two parameters will be used throughout many of these designs - it will be useful to keep their naming static across all components.

- BIT_WIDTH: The number of bits which the register modules can store. Default to 8.
- LOG_PORT_DEPTH: The $\log_2$ of the number of register modules address bits. Default to 3.

Register File

The register file contains $2^{\text{LOG_PORT_DEPTH}}$ register modules and follows a two-read, one-write format. See Figure 2.1.

- Inputs (One bit unless noted otherwise):
 - `clk_n`: Clock signal. Falling edge triggered.
 - `we`: Write enable. Enabling allows register contents to be written.
 - `Addr1`, `Addr2` (`LOG_PORT_DEPTH` bits): Addresses of which registers to read from.
 - `Addr3` (`LOG_PORT_DEPTH` bits): Write address. Selects which register to write to.
 - `wd` (`BIT_WIDTH` bits): Write data. Data to be written to the appropriate register.
- Outputs:
 - `RD1`, `RD2` (`BIT_WIDTH` bits): Parallel outputs containing the data from the selected registers.



Register 0 must always output 0s and be non-writable.



The register file is basically a memory with two output ports, each with its own address.



Use a Conditional Signal Assignment (“When/Else” Statement) to assign the read output to zero when the read address is zero.



The clock for the register file is falling edge triggered.

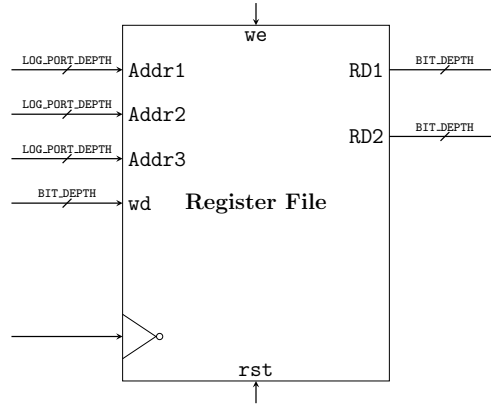


Figure 2.1: Register File

Hardware Demonstration

For the Hardware Demonstration in this exercise, only one register will be used for testing. Students will also need to keep (LOG_PORT_DEPTH) & (BIT_WIDTH) at their default values. The I/O pins to assign to the Basys3 FPGA will be as follows:

- Inputs:
 - `clk_n`: Clock signal. Falling edge triggered.
 - `we`: Write enable. Enabling allows register contents to be written.
 - `Addr1` (LOG_PORT_DEPTH): Address of which register to read from.
 - `Addr3` (LOG_PORT_DEPTH): Write address. Selects which register to write to.
 - `wd` (BIT_WIDTH): Write data. Data to be written to the appropriate register.
- Outputs:
 - `RD1` (BIT_WIDTH): Parallel output containing the data from the selected register.



Control signals can be placed on the buttons of the board

Students must demonstrate the following scenarios:

- Writing a value to an empty register, then reading that value.
- Overwriting a value in a register, then reading the new value.
- Reading from a register that has not been written to.
- Attempting to write to a register when `we` is '0'. Should fail.

Testbench

Write a testbench to ensure that the register file is working properly. The testbench should be self-checking and use a record array to store test cases. Some tests to consider are:

This testbench must use assert statements to confirm proper functionality.

Lab Procedure

1. Create a new directory and Vivado project for this exercise.
2. Write properly commented and properly formatted VHDL according to the preceding specifications.
 - a. Create a Register File entity with all input and output signals according to the hardware specification.
 - b. Create an architecture that represents the functionality of a Register File.
 - i. Create a data type that represents the register module, or memory. This should be an array of vectors, similar to a 2D array. Create a signal of this data type and instantiate each register to zero.



Remember that due to the nature of VHDL and how memory is accessed, the create **array** needs to use the keyword **to** instead of the keyword **downto**.

- ii. Create a process that is sensitive to the Register File's clock signal.
 - iii. Assign **wd** to the address specified in **Addr3** when the correct conditions are met.
 - iv. Assign the data contained in **Addr1** and **Addr2** to **RD1** and **RD2**, respectively, so that they read asynchronously.



Remember that asynchronous actions happen outside of a process, or else they will be dependent on the specified signals in the process sensitivity list.

- c. Modify the provided testbench to properly and fully test the Register File.
 1. Add more cases that test writing to and reading from every register.
 2. Ensure that Register 0 is immutable (cannot be written to).
 3. Ensure that assert statements are working properly. Assert statements should say the expected result and what result was actually obtained.
3. Test the design using a behavioral simulation.
4. Synthesize the design and test it using a Post-Synthesis Timing simulation.



Timing simulations include all the delays in the FPGA including those within the flip-flops themselves. Add a delay at the beginning of the testbench so the flip-flops have at least two clock edges before reading or writing.

5. Generate the RTL schematic that shows the Register File. (this is labeled under rtl Analysis, under open elaborated design, click schematic).
6. Implement the design and test it using a Post-Implementation Timing simulation.
7. Test the design using EDAPlayground simulation. Refer to the EDAPlayground Tutorial in the Appendix I for instructions on how to create a simulation.
8. Perform the the Hardware Demonstration, referencing the tutorial in Lab 1 if needed.
9. Demonstrate the above steps to a TA or the instructor.
10. Screen shot the appropriate waveforms for the report and ensure that they will be readable.
11. Write a full lab report describing the Register File created, specifically citing how it was designed and its functionality was verified. See this exercise's lab report deliverables for more details.

Writing A Constraints File

In the initial laboratory session, you were provided with a constraints file; however, in this subsequent lab, the task will require creating your own constraints. In the previous session, you gained insight into the typical structure of constraints files. Specifically, the command:

`"set_property PACKAGE_PIN V17 [get_portsB[0]]"` in Vivado designates the port on the device that will be driven and the voltage with:

`"set_property IOSTANDARD LVCMOS33 [get_ports B[0]],"` determining the level at which the signal will be driven.

Align your input and output signals with the respective ports on the device. For instance, in the provided example, `'B[0]'` and `'switch V17'` correlate. However, if V17 represented an LED rather than a switch, you would need to use an output signal instead. To identify which signal corresponds to each port, look at the labels on your board. The port naming scheme, is not immediately intuitive, so carefully read it.

You will also need a clock in this lab, the following code should add a clock to your constraints file, do not try to assign your clock to a switch, there is a specific component on the board that will be more than sufficient to be your clock. `set_property PACKAGE_PIN W5 [get_ports clk] set_property IOSTANDARD LVCMOS33 [get_ports clk] create_clock -add -name sys_clk_pin -period 10.00 -waveform 0 5 [get_ports clk]`

What is the clock: The clock is a component on every PC, that is used to synchronize the execution of instructions on the device.

Exercise 2: Register File

Student's Name: _____

Section: _____

PreLab		Point Value	Points Earned	Comments
PreLab	MIPs Questions	10		
	Draw a register, and explain how it works	5		
	Explain what the purpose of the register file stage is in the MIPS	5		
	Testbench Cases	5		

Demo		Point Value	Points Earned	Date
Demo	Behavioral Simulation	10		
	Post-Synthesis Timing Simulation	5		
	RTL Schematic	5		
	Post-Implementation Timing Simulation	5		
	Implementation Utilization Report	5		
	EDAPlayground Behavioral Waveform	5		
	Hardware Demonstration	10		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

Exercise 2: Register File

Report		Point Value	Points Earned	Comments
Abstract		5		
Design Methodology	Discussion of Register Module Functionality	5		
	Discussion of Register File Functionality	5		
Results & Analysis	Behavioral Waveform	3		
	Post-Implementation Waveform	3		
	EDAPlay-ground Waveform	4		
	RTL Schematic, breakdown	5		
	Results Description	5		
Conclusion		5		
Source Code (Style)		5		
Total for prelab, demo, and report		100		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

In-Lab Deliverables

Register File

- VHDL Code for generic, 32-bit Register File.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design uses constants **BIT_WIDTH** and **LOG_PORT_DEPTH** with correct values.
- VHDL Testbench Code for generic, 32-bit Register File.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must implement seven additional cases as stated in the Prelab section of this exercise.
 - Testbench must read from and write to different registers.
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL Testbench must successfully perform behavioral, post-synthesis timing, and post-implementation timing simulations.
- Synthesis schematic for Register File.
- Implementation Utilization Report for Register File.
- Behavioral waveform generated in EDAPlayground.
- Hardware Demonstration of 32-bit Register File.

Lab Report Deliverables

Cover Page

- Proper cover page that includes exercise number and name as well as student's name, lab section, lab instructor, teaching assistants, and lecture instructor.

Abstract

- Objective of the exercise.
- Summary of what was done.
- How the design was tested.
- Results of exercise (successful or unsuccessful).

Design Methodology

- Discussion of Register Module and its functionality. Discussion should include how the number of registers was calculated and why. The size of each register should also be discussed. It should also be mentioned that the Register Module is in the form of a “2-D Array”
- Discussion of Register File and its functionality. Discussion should include how the Register File uses its inputs to determine its outputs. Register zero being immutable should also be mentioned.

Results and Analysis

- Waveforms and discussion of behavioral and post-implementation timing simulations. Cite specific examples of proof that the waveforms are correct. Stating that the wave matched the expected results with no test cases cited does not explain what the expected results were, or how they matched the testbench.
- Waveform generated on EDAPlayground and comparison to behavioral waveform generated in Vivado.
- Discussion of test cases.

Conclusion

- Restate objective and summary of exercise.
- State the final results of the exercise.
- Reflections of lessons learned and discussion of future use of what was learned.

Other

- Report should be original and follow digital system design report guidelines.

Other Deliverables

- All VHDL code must be properly commented and have proper comment headers that detail the student’s name, lab section, and file description.
- All submissions must follow the submission guidelines detailed within the preliminary matter of this manual.
- **ALL VHDL CODE MUST BE SUBMITTED TO MYCOURSES AS .TXT FILES TO RECEIVE A GRADE.**

Exercise 3 Instruction Fetch and Decode Stages

Objective

In this exercise, students design and create the Instruction Fetch and Decode stages of a pipelined MIPS architecture. The Instruction Fetch will require making several different modules and connecting them into a single Instruction Fetch stage. The Instruction Fetch stage is responsible for bringing the program instructions from memory into the processor. The Instruction Decode stage is responsible for parsing the instructions passed from the Instruction Fetch stage into commands which the Execute stage can then put into action.

The full MIPS datapath will look similar to Figure 5.1.

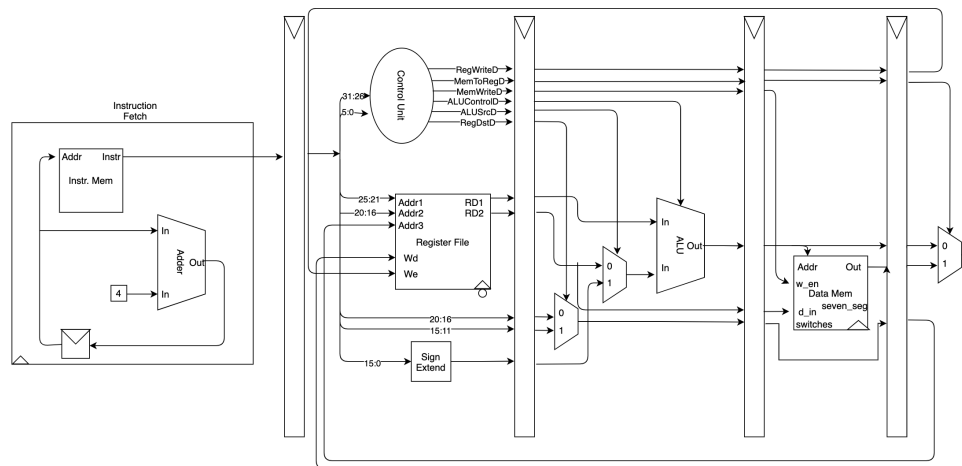


Figure 3.1: Pipelined MIPS Datapath

This exercise will focus on designing the first two stages in the Figure.

Pre-lab Activities

- Describe in a brief paragraph the role of the Instruction Fetch stage in a MIPS processor.
- Describe in a brief paragraph the role of the Instruction Decode stage in a MIPS processor.
- Fill out the testbench shell for the Instruction Fetch Stage from the MyCourses.
- Create a block diagram for the wrapper of the Instruction Memory and for the Instruction Decode. In future sections descriptions of Instruction Fetch and Instruction Memory will help create the wrappers.



A block diagram of a wrapper shows how the input/ output signals from the overall stage connect to the input/ output signals of the internal components. Be sure to include the number of bits on each signal.



Signals names often change. In the wrapper diagram, something like “RegWriteD” will make sense at the top level, but internally it is just called RegWrite.



Signals change names because convention is to shorten names like WriteData to Wd, and at the top level RegWriteData makes sense. In the wrapper these signals must be bridged so that the correct lines get placed and account for signals with different names.



A wrapper level does not have to be completely structural. Simple code like the sign extend can be written in instruction decode file.

- Print the sign-off sheet.

Hardware Specification - Fetch

Instruction Memory

The Instruction Memory module will hold all of the instructions which will be fetched by the Instruction Fetch. This memory can be designed to be either byte addressable, or word addressable, and should hold 1024 bytes.

In order to output words when the memory is only byte addressable, the memory will need to concatenate the value being accessed and the 3 bytes following it.

- Inputs (One bit unless noted otherwise)
 - **addr** (28 bits) Address in memory to read from
- Outputs (One bit unless noted otherwise)
 - **d_out** (32 bits) The instruction read from memory



If an address greater than 1023 is provided, the memory should return all 0's. This is in place because of memory limitations for the FPGA. In VHDL, this can be done in a process using conditional logic. Be sure to check that the address is in the correct form to be compared to an integer.

Instruction Fetch

The Instruction Fetch module will incorporate the Instruction Memory module. The fetch will retrieve an instruction from memory at the address determined by the program counter, which should increment by 4 each clock cycle and over time fetch all instructions from the memory.

- Inputs (One bit unless noted otherwise)
 - `clk` The system clock
 - `rst` Active high reset. Asynchronous.
- Outputs (One bit unless noted otherwise)
 - `Instruction` (32 bits) The instruction which was fetched from memory

Hardware Specification - Decode

Instruction Decode

The Instruction Decode stage takes a 32-bit instruction and decodes that code into its components. It also accesses all necessary registers for that instruction.

- Inputs (One bit unless noted otherwise)
 - `clk` The system clock
 - `Instruction` (32 bits) The instruction being decoded
 - `RegWriteAddr` (5 bits) The address of the register being written to.
 - `RegWriteData` (32 bits) Data to be written into the Register File.
 - `RegWriteEn` Register File write enable.
- Outputs (One bit unless noted otherwise)
 - `RegWrite` Control bit. Set if the instruction requires register writing.
 - `MemtoReg` Control bit. Set if the instruction requires reading from Memory.
 - `MemWrite` Control bit. Set if the instruction requires writing to Memory.
 - `ALUControl` (4 bits) Op-code specific to the ALU
 - `ALUSrc` Control bit. Set if the ALU will use an immediate.
 - `RegDst` Control bit. Determines which register will be used as the destination register.
 - `RD1` (32 bits) Register File output 1
 - `RD2` (32 bits) Register File output 2
 - `RtDest` (5 bits) Address of Rt from instruction (used with `RegDst` in a later stage).
 - `RdDest` (5 bits) Address of Rd from instruction (used with `RegDst` in a later stage).
 - `ImmOut` (32 bits) Sign-extended immediate for I-Type instructions.

Control Unit

The Control Unit is responsible for breaking the 32 bit instructions into multiple other signals for use in later stages. The information necessary for this is broken down in the “MIPS Instructions” section.

- Inputs (One bit unless noted otherwise)

- **Opcode** (6 bits) The opcode of the instruction
- **Funct** (6 bits) The function of the instruction
- **Outputs** (One bit unless noted otherwise)
 - **RegWrite** Control bit. Set if the instruction requires register writing.
 - **MemtoReg** Control bit. Set if the instruction requires reading from Memory.
 - **MemWrite** Control bit. Set if the instruction requires writing to Memory.
 - **ALUControl** (4 bits) Op-code specific to the ALU
 - **ALUSrc** Control bit. Set if the ALU will use an immediate.
 - **RegDst** Control bit. Determines which register will be used as the destination register.

Register File

The register file is used to store data which will be accessed frequently or does not need to be permanently stored. The register at address 0 should always produce 0s at the output.

- **Inputs** (One bit unless noted otherwise)
 - **clk_n** The system clock
 - **we** Write enable
 - **Addr1** (5 bits) Address for reading a register into RD1
 - **Addr2** (5 bits) Address for reading a register into RD2
 - **Addr3** (5 bits) Address for writing data
 - **WriteData** (32 bits) Data to be written into the Register File
- **Outputs** (One bit unless noted otherwise)
 - **RD1** (32 bits) Register File output 1
 - **RD2** (32 bits) Register File output 2

MIPS Instructions

The MIPS instruction set is made of a number of 32-bit instructions, which can be broken down by the Instruction Decode stage into smaller segments for different components to handle. There are three types of instructions: R-type (Register Type), I-type(Immediate Type), and J-type (Jump Type). Each has a slightly different format for the instructions. The type of instruction is determined by the op-code of each instruction, which is always the first 6 bits. In this course, students will implement Register and Immediate Type instructions, but not Jump Type instructions.

Register Type Register Type instructions act upon 3 different registers, a source, target, and destination. The function, which is the final 6 bits, determines which ALU operation is to be executed with these registers. The shift amount (**sh.amt**) is the amount to shift by when using a shift command.

The Op-code for an R-Type instruction is “000000”

Table 3.1: R-Type

Op-code	rs	rt	rd	sh_amt	function
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Table 3.2: ALU Operations for R-Type

Function	Code
ADD	100000
AND	100100
MULTU	011001
OR	100101
SLL	000000
SRA	000011
SRL	000010
SUB	100010
XOR	100110



Technically the SLL and SRL implemented in these exercises are SLLV and SRLV instructions (Shift Left/Right Logical Variable), which use the value in rs, rather than the shift amount. The information for shift amount is included here, but will not actually be used. The SRA being implemented works the same way, but in a proper implementation of MIPS the SRAV instruction does not exist - it is only used here to simplify the design of the ALU in a future instruction.

Immediate Type Immediate Type instructions are functions which act upon a number stored in a register and a 16 bit immediate. Unlike R-Type operations, the instruction op-code determines both that this is an I-Type instruction as well as what operation is to be performed.

Table 3.3: I-Type

Op-code	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Table 3.4: ALU Operations for I-Type

Function	Code
ADDI	001000
ANDI	001100
ORI	001101
XORI	001110
SW	101011
LW	100011

ALU Codes The ALUOp output is only 4 bits; however, the part of the op-code which determines the ALU operation is 6 bits. Table 3.5 contains the 4 bit op-codes for the ALU which will be needed

for R and I Type Instructions.

Table 3.5: 4-Bit ALU Op-Codes

Function	Code
ADD/ADDI/SW/LW	0100
AND/ANDI	1010
MULTU	0110
OR/ORI	1000
XOR/XORI	1011
SLL	1100
SRA	1110
SRL	1101
SUB	0101

Lab Procedure

Part 1 — Instruction Fetch

1. Create a new Project in Xilinx Vivado for the exercise
2. Write properly commented and formatted VHDL according to the Hardware Specification.
 - A. Create a new VHDL file for the Instruction Memory.
 - B. Implement the Instruction Memory according to the Hardware Specification.
 - i. Create an entity for the Instruction Memory with the specified inputs and outputs.
 - ii. Create an architecture that represents the functionality of the Instruction Memory.
 - a. Similar to the Register File's register module, create a data type that represents memory that will be used to hold instructions.
 - b. Create a process with a proper sensitivity list for the Instruction Memory.
 - c. Ensure that the address is not bounds. If it is, set the data out to zero. Otherwise, set the output to the correct data.



Remember that the Instruction Memory is either byte or word addressable. Therefore, when accessing data, the current address and the next three addresses need to be accessed and concatenated to create the correct output.

- C. Create a new VHDL file for the Instruction Fetch.
 - i. Create an entity and architecture that represents the Instruction Memory according to the Hardware Specification.
 - ii. Instantiate the previously created Instruction Memory as a component.
 - iii. Map the signals of the Instruction Memory to the Instruction Fetch.
 - iv. Create a process with a proper sensitivity list for the Instruction Fetch.
 - v. The Fetch reset should happen asynchronously and the PC (Program Counter) update should happen on the rising edge of the clock.

3. Modify the testbench created in the prelab.
 - A. Add **at least three** more test cases and incorporate assert statements to make the testbench self-checking. These tests are just to ensure that Instruction Fetch is properly reading the Instruction Memory, so the test cases do not need to be actual instructions.
 - B. Add a test case that tests an instruction that would cause the Instruction Memory to access out of bounds memory.
 - C. Add a test case that tests an instruction that would not initially be out of bounds, but accessing the following bytes in the word would cause an out of bounds error.



The first instruction should be seen on the output twice. While in reset the memory should always return the value held at address 0.

4. Test the design with a Behavioral Simulation.
5. Synthesize the design and test it using a Post-Synthesis Timing Simulation.
6. Implement the design and test it using a Post-Implementation Timing Simulation.
7. Test the design using ModelSim. Refer to the ModelSim Tutorial in the Appendix J for instructions on how to create a simulation.
8. Demonstrate the above steps to a TA or Lab Instructor — ensure there are no warnings in the Synthesis and no errors in the Implementation.
9. Screenshot the appropriate waveforms for the report, making sure they are readable.

It is recommended that Part 1 is completed in the first week of this exercise.

Part 2 — Instruction Decode

A working Register File is needed for this part of the exercise.

Exercise 2 should be completed before attempting the Instruction Decode.

1. Create a new Project in Xilinx Vivado for the exercise.
2. Write properly commented and formatted VHDL according to the preceding specifications.
 - A. Create a new VHDL file for the Control Unit.
 - i. Create an entity and architecture that represents the Control Unit according to the Hardware Specification.
 - ii. Create a process for **each** control signal created by the Control Unit.



The Control Unit uses many processes to create control signals. It is good practice to use a separate process for each signal.

- B. Create a new VHDL file for the Instruction Decode.
 - i. Create an entity and architecture that represents the Instruction Decode according to the Hardware Specification.
 - ii. Instantiate the previously created Control Unit as a component.
 - iii. Instantiate the Register File created in Exercise 2 as a component.
 - iv. Parse the instruction into different signals necessary for the Control Unit, Register File, and Decode output.
 - v. Using the Decode inputs and parsed instruction, map the instantiated components to the Instruction Decode according to the Hardware Specification.
3. Modify the given testbench shell. Add a new R type and a new I type test case and incorporate assert statements to make the testbench self-checking.
4. Test the design with a Behavioral Simulation.
5. Synthesize the design and test it using a Post-Synthesis Timing Simulation.
6. Implement the design and test it using a Post-Implementation Timing Simulation. This should lead to an error which refers to “overutilization”. To solve this issue, follow the instructions in the section below.
7. Demonstrate the above steps to a TA or Lab Instructor — ensure there are no warnings in the Synthesis and no errors in the Implementation.
8. Screenshot the appropriate waveforms for the report, making sure they are readable.

Overutilization Error

If an “overutilization” error is encountered during the Implementation step, students should refer to Appendix K for the solution to this error.

Exercise 3: Instruction Fetch and Decode Stages

Student's Name: _____

Section: _____

PreLab		Point Value	Points Earned	Comments
PreLab	Fetch TB Shell	2		
	Fetch Stage Wrapper	2		
	Fetch stage description	2		
	Decode stage description	2		
	Decode Stage Wrapper	2		

Demo		Point Value	Points Earned	Date
Part 1 – Fetch	Behavioral Simulation	6		
	No Synthesis Warnings	4		
	Post-Synthesis Timing Simulation	3		
	No Implementation Warnings	4		
	Post-Implementation Timing Simulation	3		
	ModelSim Behavioral Waveform	5		
Part 2 – Decode	Behavioral Simulation	7		
	No Synthesis Warnings	5		
	Post-Synthesis Timing Simulation	4		
	No Implementation Warnings	5		
	Post-Implementation Timing Simulation	4		

To receive any grading credit students must earn points for both the demonstration and the report.

Exercise 3: Instruction Fetch and Decode Stages

Report		Point Value	Points Earned	Comments
Abstract		3		
Design Methodology	Discussion of Instruction Memory Functionality	3		
	Discussion of Instruction Fetch Functionality	3		
	Discussion of Instruction Decode Functionality	3		
	Discussion of significance Fetch & Decode in the larger MIPS Design	5		
Results & Analysis	Fetch Behavioral Waveform	3		
	Fetch Results Description	4		
	Decode Behavioral Waveform	3		
	Decode Results Description	4		
Conclusion		3		
Source Code (Style)		3		
Total for prelab, demo, and report		100		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

In-Lab Deliverables

Part 1 — Instruction Memory and Instruction Fetch

- VHDL Code for Instruction Memory.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design properly checks for out of bounds memory accesses.
 - Design properly returns 32-bit word data if memory access is valid.
 - Memory is properly instantiated with instructions to be fetched.
 - Unused memory is properly instantiated to zero.
- VHDL Code for Instruction Fetch.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design properly instantiates and incorporates the Instruction Memory.
 - Design properly resets the program counter asynchronously.
 - Design properly updates the program counter on the rising-edge of the clock.
- VHDL Testbench Code for Instruction Fetch.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must implement three general test cases in addition to the test cases given in the Prelab.
 - Testbench must implement two edge cases testing memory bounds, specified in Procedure step 3.
 - Testbench **does not** have to implement real instructions as test cases.
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL Testbench must successfully perform behavioral, post-synthesis timing, and post-implementation timing simulations.
- Synthesis and Implementation must not have any warnings.
- Behavioral waveform generated in ModelSim.

It is recommended that Part 1 is completed in the first week of this exercise.

In-Lab Part 2 Deliverables begin on next page.

Part 2 — Control Unit and Instruction Decode

A working Register File from Exercise 2 is needed to complete this Part of the Exercise.

- VHDL Code for Control Unit.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design properly uses unique processes for each control signal.
 - Correct control signals are returned for each test case.
- VHDL Code for Instruction Decode.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design properly instantiates and incorporates the Control Unit.
 - Design properly instantiates and incorporates the Register File.
 - Design properly parses the instruction and gives the correct values for each output.
- VHDL Testbench Code for Instruction Decode.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must implement an R-Type and I-Type instruction. Instructions need to be real instructions that would be used in MIPS assembly.
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL Testbench must successfully perform behavioral, post-synthesis timing, and post-implementation timing simulations.
- Synthesis and Implementation must not have any warnings.

Lab Report Deliverables

Cover Page

- Proper cover page that includes exercise number and name as well as student's name, lab section, lab instructor, teaching assistants, and lecture instructor.

Abstract

- Objective of the exercise.
- Summary of what was done.
- How the design was tested.
- Results of exercise (successful or unsuccessful).

Design Methodology

- Discussion of the Instruction Memory and its functionality. Discussion should include how the Instruction Memory's memory module was created and the size of the data at each address should be discussed (e.g. discussion of word addressable design). Out of bounds memory access checking should also be discussed.
- Discussion of the Instruction Fetch and its functionality. Discussion should include how the Instruction Fetch asynchronously resets and updates the program counter. Discussion should also include how the Instruction Memory was incorporated.
- Discussion of the Control Unit and its functionality. Discussion should include how each control signal was given a unique process to determine its output. Discussion should include explanation of how specific instruction functions (and their corresponding operations) determine a specific ALU code (E.g. ADD/ADDI/SW/LW all use the ALU code for addition).
- Discussion of the Instruction Decode and its functionality. Discussion should include how the Register File and Control Unit are incorporated and how the instruction is parsed to generate a decoded output.
- Discussion of how the Instruction Fetch and Instruction Decode are connected.

Results and Analysis

- Instruction Fetch Waveforms and discussion of behavioral and post-implementation timing simulations. Cite specific examples of proof that the waveforms are correct. Stating that the wave matched the expected results with no test cases cited does not explain what the expected results were, or how they matched the testbench.
- Instruction Fetch Waveform generated on ModelSim and comparison to behavioral waveform generated in Vivado.
- Discussion of Instruction Fetch test cases.
- Instruction Decode Waveforms and discussion of behavioral and post-implementation timing simulations.
- Discussion of Instruction Decode test cases.

Conclusion

- Restate objective and summary of exercise.
- State the final results of the exercise.
- Reflections of lessons learned and discussion of future use of what was learned.

Other

- Report should be original and follow digital system design report guidelines.

Other Deliverables

- All VHDL code must be properly commented and have proper comment headers that detail the student's name, lab section, and file description.
- All submissions must follow the submission guidelines detailed within the preliminary matter of this manual.
- **ALL VHDL CODE FOR BOTH PARTS 1 AND 2 MUST BE SUBMITTED TO MYCOURSES AS .TXT FILES TO RECEIVE A GRADE.**

Exercise 4 Execute Stage

Objective

In a MIPS processor, once an instruction has been fetched from memory and decoded it needs to be executed. The execute stage consists of an ALU and several multiplexers, which determine how the instruction being passed in is executed. For the purposes of this design, these multiplexers will not need to be explicitly created. In this exercise, the ALU that was begun in an earlier exercise will need to be completed with a Ripple-Carry full adder/subtractor and a Multiplier. Next, once the ALU has all of the required functionality, the complete execute stage must be created so that it can properly interface with the previous stages.

The full MIPS datapath will look similar to Figure 5.1.

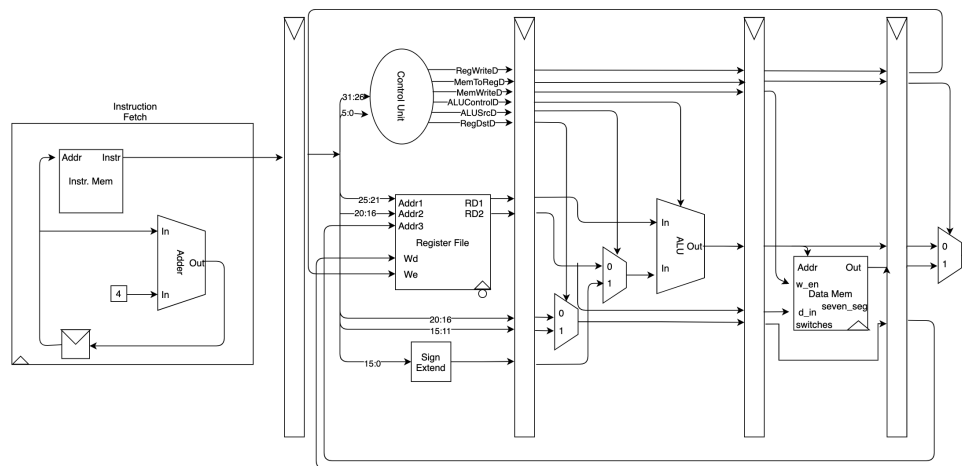


Figure 4.1: Pipelined MIPS Datapath

This exercise will focus on designing the middle stage in the Figure.

Pre-lab Activities

- Draw a block diagram of the complete ALU, including the Ripple-Carry Adder/Subtractor and Multiplier



This must include the internal structure of the Adder and Multiplier

- Create a block diagram for the wrapper of the Execute Stage
- Become familiar with the functionality of a multiplier. Consider what might need to be modified to make it a generic size.
- Fill out the testbench shell from MyCourses
- Print the sign of sheet

Hardware Specification

ALU

The full ALU has several different components. Most of these were created in previous exercises; however, there are two more components which need to be added for full functionality: the adder/-subtractor and the multiplier.

Once all operations have been created, Table 4.1 should be used to determine which operation is being performed.

- Parameters
 - **N**: The bit depth of the signals entering the ALU. Default to 32.
- Inputs (One bit unless noted otherwise)
 - **in1** (N bits): First input to the ALU
 - **in2** (N bits): Second input to the ALU
 - **control** (4 bits): The ALU Op-code
- Outputs (One bit unless noted otherwise)
 - **out1** (N bits): Output of the ALU

Table 4.1: 4-Bit ALU Op-Codes

Function	Code
ADD/ADDI/SW/LW	0100
AND/ANDI	1010
MULTU	0110
OR/ORI	1000
XOR/XORI	1011
SLL	1100
SRA	1110
SRL	1101
SUB	0101

Execute Stage

The execute stage of the MIPS processor takes the outputs from the Instruction Decode stage and executes the appropriate instruction. A major part of this stage is the ALU - however, there are other components and signals involved.

- Inputs (One bit unless noted otherwise)
 - **RegWrite**: Determines whether the instruction being processed writes to a register.
 - **MemtoReg**: Determines whether the instruction being processed reads from memory.

- **MemWrite**: Determines whether the instruction being processed writes to memory.
- **ALUControl** (4 bits): The ALU Opcode.
- **ALUSrc**: If set then the ALU uses the immediate, otherwise it uses two registers.
- **RegDst**: Determines which register is the destination register of the instruction.
- **RegSrcA** (32 bits): Data stored in the first register being read.
- **RegSrcB** (32 bits): Data stored in the second register being read.
- **RtDest** (5 bits): Address of rt in the instruction.
- **RdDest** (5 bits): Address of rd in the instruction.
- **SignImm** (32 bits): The sign extended immediate of the instruction.
- **Outputs** (One bit unless noted otherwise)
 - **RegWriteOut**: Control bit passthrough.
 - **MemtoRegOut**: Control bit passthrough.
 - **MemWriteOut**: Control bit passthrough.
 - **ALUResult** (32 bits): The value output by the ALU.
 - **WriteData** (32 bits): The data to be written into memory.
 - **WriteReg** (5 bits): The address of the register being written to.

Ripple Carry Full Adder

The full adder has the following features:

- **Functions**:
 - Multi-bit addition.
 - Multi-bit subtraction. Thus all multi-bit inputs and outputs are in two's complement.
- **Parameters**:
 - **N**: The number of bits of the two addends. Default to 32.
- **Inputs**:
 - **A** (N bits): First addend, or minuend if subtracting
 - **B** (N bits): Second addend, or subtrahend if subtracting
 - **OP**: Op-code specific to Adder/Subtractor. 0 - Addition, 1 - Subtraction.
- **Outputs**:
 - **Sum** (N bits): Sum, or difference if subtracting

The ripple carry full adder must be structural, however, the full adders it is made up of can be behavioral, dataflow, or structural. Use the **for generate** syntax to instantiate the individual full adders inside of the ripple carry adder.

Multiplier

The multiplier has the following features:

- Functions:
 - Multi-bit multiplication.
- Parameters:
 - N: The output length, in bits. Default to 32.
- Inputs:
 - A (N/2 bits): First factor.
 - B (N/2 bits): Second factor.
- Outputs:
 - Product (N bits): Product. Note that this is twice as long as the factors.

Two examples of a multiplier which could be used for this exercise are below. Figure 4.3 shows a four-bit carry-save multiplier. Note that the rightmost adder could be a half adder, as the carry in is set to zero. This multiplier functions in a similar way to humans performing multiplication with two decimal numbers on paper. In order to implement such a multiplier, consider using an array of `std_logic_vectors` to hold the intermediate results such as the results of the AND operation, sums, and carry outs. The use of the `+` and `*` operators is not allowed.

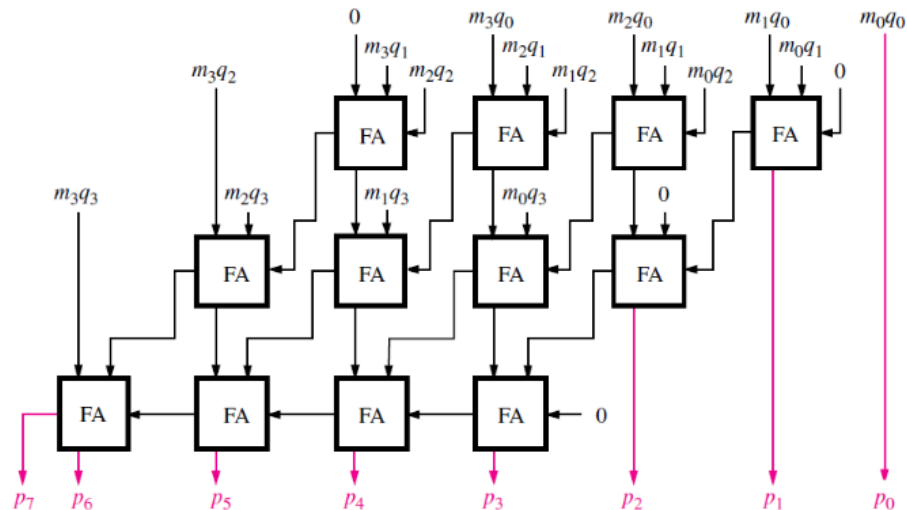


Figure 4.2: 4-bit Carry-Save Multiplier

Figure 4.3 shows a ripple carry multiplier. The multiplication process starts from the least significant bits, and each partial product is added to the result with a shifting operation. The “ripple carry” term refers to the fact that the carry-out from each addition stage ripples through to the

next, influencing the subsequent additions. While a ripple carry multiplier is simple and easy to implement, it tends to have a longer propagation delay compared to the carry-save multiplier.

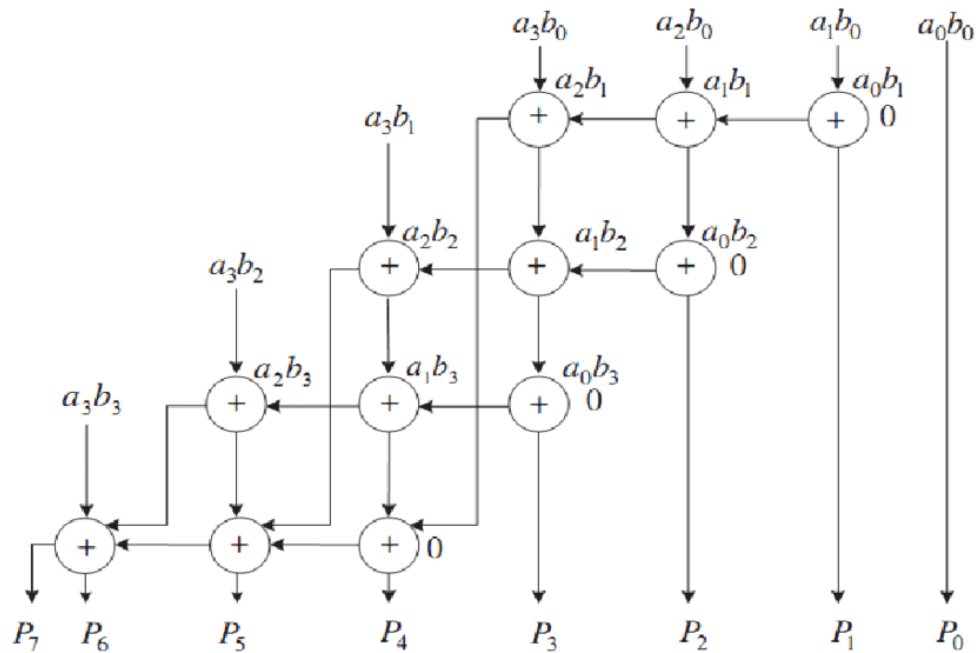


Figure 4.3: ripple carry Multiplier

Hardware Demonstration

For the Hardware Demonstration in this exercise the full functionality of the ALU will be tested. Students will need to reduce the number of bits for the inputs to 4 to program the Basys3. For full credit students must test the following:

- 4 Addition cases
- 4 Subtraction cases
- 4 Multiplication cases
- 1 Bitwise AND case
- 1 Bitwise OR case
- 1 Bitwise XOR case
- 1 Bitwise Shift Left
- 1 Bitwise Shift Right
- 1 Bitwise Shift Right Arithmetic

Procedure

Part 1 - Ripple-Carry Adder & Execute Stage

1. Create a new Project in Xilinx Vivado for the exercise.
2. Write properly commented and formatted VHDL according to the preceding specifications. The multiplier does not need to be completed in this part, the Adder/Subtractor and execute stage shell are required for Part 1.



For an N bit Ripple Carry Adder/Subtractor, use the VHDL keyword **for generate** to generate N full adders.

3. Create an ALU testbench by modifying the testbench created in the prelab. Add any possible missing cases and incorporate assert statements to make the testbench self-checking. Test the full ALU **except the Multiplier**.
 - Create at least three general test cases for **each** operation. In addition to the general cases, create two edge cases demonstrating overflow and underflow for the created Ripple Carry Adder/Subtractor.
4. Test the ALU with a Behavioral Simulation.
5. Synthesize the ALU and test it using a Post-Synthesis Timing Simulation.
6. Implement the ALU and test it using a Post-Implementation Timing Simulation.
7. Create the Execute Stage for the MIPS Datapath - make sure the ALU is the full size (32 bits) when adding it to the stage.
8. Write an Execute stage testbench which properly tests the current functionality of the full Execute Stage.
 - Create one case for each operation. A case for the multiplier does not need to be done for Part 1.
9. Test the Execute stage design with a Behavioral Simulation.
 - It is not required to synthesize or implement the design at this stage, though it is recommended to ensure the correct functionality.
10. Demonstrate the above steps to a TA or Lab Instructor.
11. Screenshot the appropriate waveforms for the report, making sure they are readable.

It is highly recommended that Part 1 is completed within the first week of the lab.

Part 2 - Multiplier

1. Create a VHDL design of the Multiplier.
2. Create a testbench for the Multiplier which properly tests the functionality of the Multiplier.
 - Create at least seven general cases.
 - Create an edge case where an operand is zero.
 - Create an edge case where both operands are the maximum value.
 - Create an edge case where overflow occurs without using maximum value operands.
 - Students may be asked to implement more test cases by their lab instructor or TAs, in which those cases should also be tested.
3. Test the Multiplier design with a Behavioral Simulation.
4. Add the Multiplier to the rest of the ALU.
5. Test the completed ALU design with a Behavioral Simulation by modifying the testbench created for the ALU in Part 1.
 - Create three general cases for multiplication.
 - Create two edge cases for multiplication.
 - Cases from the Multiplier Testbench can be re-used for the Full ALU.
6. Synthesize the complete ALU design and test it using a Post-Synthesis Timing Simulation.
7. Implement the complete ALU design and test it using a Post-Implementation Timing Simulation.
8. Perform the Hardware demonstration to a TA.
9. Behaviorally test the completed Execute Stage with a completed ALU including the multiplier.
10. Demonstrate the above steps to a TA or Lab Instructor.
11. Screenshot the appropriate waveforms for the report, making sure they are readable.



The Multiplier will most likely use multiple **for generate** statements, making it somewhat difficult to picture without physically drawing it out. It is recommended to draw out a smaller multiplier (e.g. 4x4 multiplier) out on paper, think about what intermediate signals are necessary for the design, and then attempting to write the code.



Utilize the RTL schematic of your multiplier design to trace the flow of the bits. It can be used to help visualize the design and to help debug.

Exercise 4: Execute Stage

Student's Name: _____

Section: _____

PreLab		Point Value	Points Earned	Comments
PreLab	ALU Block Diagram	3		
	Execute Stage Wrapper	2		
	ALU Testbench Shell	5		

Demo		Point Value	Points Earned	Date
Part 1 – Adder& Execute Stage	ALU Behavioral Simulation	10		
	ALU Post-Synthesis Timing Simulation	5		
	ALU Post-Implementation Timing Simulation	5		
	Execute Stage Behavioral Simulation	5		
Part 2 – Multiplier	Mult Behavioral Simulation	5		
	Full ALU Behavioral Simulation	2		
	Post-Synthesis Timing Simulation	5		
	Post-Implementation Timing Simulation	5		
	Hardware Demonstration	6		
	Completed Execute Stage Behavioral Simulation	5		

To receive any grading credit students must earn points for both the demonstration and the report.

Exercise 4: Execute Stage

Report		Point Value	Points Earned	Comments
Abstract		5		
Design Methodology	ALU and Execute Block Diagrams	5		
	Discussion of significance and use of the execute stage in the larger MIPS Design	5		
	Discussion of Functionality	5		
Results & Analysis	ALU and Execute Behavioral Simulations	5		
	Discussion of Results	4		
Conclusion		5		
Source Code (Style)		5		
Total for prelab, demo, and report		100		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

In-Lab Deliverables

Part 1 — Ripple Carry Adder/Subtractor, ALU, and Execute Stage

It is highly recommended that Part 1 is completed in the first week of this exercise.

- VHDL Code for Full Adder.
 - Full adder is created and used for structural Ripple Carry Adder/Subtractor.
- VHDL Code for Ripple Carry Adder/Subtractor.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design is structural and uses full adders, as specified in the Hardware Specification.
 - Design uses VHDL keyword **for generate** to instantiate individual full adders.
- VHDL Code for ALU.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design uses the correct ALU op-codes for functions listed in the Hardware Specification.
 - Design instantiates and uses the previously created AND, OR, XOR, SLL, SRA, and SRL components for their respective functions.
 - Design instantiates and uses the Ripple Carry Adder/Subtractor for addition and subtraction based functions.
- VHDL Testbench Code for ALU.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must implement at least three general test cases for each operation (two for each should have been done for the prelab).
 - Testbench must implement at least two cases demonstrating overflow and underflow for the Ripple Carry Adder/Subtractor.
 - Testbench **does not** have to have multiplication cases for Part 1.
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL Testbench Code for Execute Stage.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must implement at least one general test case for each operation.
 - Testbench **does not** have to have multiplication cases for Part 1.
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL ALU and Execute Stage Testbenches must successfully perform behavioral, post-synthesis timing, and post-implementation timing simulations without synthesis and implementation warnings.

Part 2 — Multiplier, Full ALU, and Full Execute Stage

- VHDL Code for Multiplier.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design uses the correct data for operands A and B (e.g. uses least significant bits instead of most significant).
 - Design uses VHDL keyword **for generate** to instantiate individual full adders (or half adders, if the student decides to take that approach).
 - Design does **NOT** use + or * operators. Using + or * operators will result in loss of credit.
- VHDL Code for Full ALU.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design expands upon ALU from Part 1.
 - Design instantiates and uses the Multiplier for multiplication.
- VHDL Testbench Code for Multiplier.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must implement at least seven general multiplication cases.
 - Testbench must implement at least one case where one of the operands is zero.
 - Testbench must implement at least one case where both operands are the maximum value.
 - Testbench must implement at least one case where overflow is achieved without using the maximum value for the MSB.
 - Testbench must implement any other test cases specified by the lab instructors or teaching assistants in lab.
- VHDL Testbench Code for Full ALU.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench expands upon ALU Testbench from Part 1 by adding at least five multiplication cases (three general cases, two edge cases).
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL Testbench Code for Full Execute Stage.
 - Testbench must be self checking.
 - Testbench must use a record array to contain and stimulate test cases.
 - Testbench must implement at least one general test case for each operation.
 - Testbench must have user friendly assert statements that state the expected and actual results.
- VHDL Multiplier, Full ALU, and Full Execute Stage Testbenches must successfully perform behavioral, post-synthesis timing, and post-implementation timing simulations without synthesis and implementation warnings.
- Hardware Demonstration of cases listed in Hardware Demonstration section.

Lab Report Deliverables

Cover Page

- Proper cover page that includes exercise number and name as well as student's name, lab section, lab instructor, teaching assistants, and lecture instructor.

Abstract

- Objective of the exercise.
- Summary of what was done.
- How the design was tested.
- Results of exercise (successful or unsuccessful).

Design Methodology

- ALU and Execute Stage Block Diagrams. Must be properly labeled, introduced, and explained.
- Discussion of ALU and its functionality. Discussion should be similar to that of Exercise 1, but should detail use of op-code and new components.
- Discussion of Ripple Carry Adder/Subtractor and Multiplier as part of ALU. Discussion should detail the design of the Ripple Carry Adder/Subtractor and Multiplier, but should not go into fine details (E.g. explain the multiplier design of choice, but do not talk about signals).
- Discussion of Execute Stage and its functionality. Discussion should include explanation of input and output signals. Passthrough signals not used by the Execute Stage should be mentioned but do not need to be explained in detail.

Results and Analysis

- ALU Waveforms and discussion of behavioral and post-implementation timing simulations. Cite specific examples of proof that the waveforms are correct. Stating that the wave matched the expected results with no test cases cited does not explain what the expected results were, or how they matched the testbench.
- Discussion of ALU test cases.
- Execute Stage Waveforms and discussion of behavioral and post-implementation timing simulations.
- Discussion of Execute Stage test cases.

Conclusion

- Restate objective and summary of exercise.
- State the final results of the exercise.
- Reflections of lessons learned and discussion of future use of what was learned.

Other

- Report should be original and follow digital system design report guidelines.

Other Deliverables

- All VHDL code must be properly commented and have proper comment headers that detail the student's name, lab section, and file description.
- All submissions must follow the submission guidelines detailed within the preliminary matter of this manual.
- **ALL VHDL CODE FOR BOTH PARTS 1 AND 2 MUST BE SUBMITTED TO MYCOURSES AS .TXT FILES TO RECEIVE A GRADE.**

Exercise 5 Memory & Writeback Stage

Objective

In this exercise, the Memory and Writeback stages of the MIPS Datapath will be translated from Verilog to VHDL.

The purpose of the Memory stage is to offer memory to the system which can be used for longer term storing and loading. This is necessary, as registers are intended to temporarily hold values for computation, but not act as long term storage. The Writeback stage redirects a number of signals back into previous stages of the system. This exercise is intended to give a good understanding of the Verilog Hardware Description Language, and how it can be translated back to VHDL.

The full MIPS datapath, once completed, will look similar to Figure 5.1.

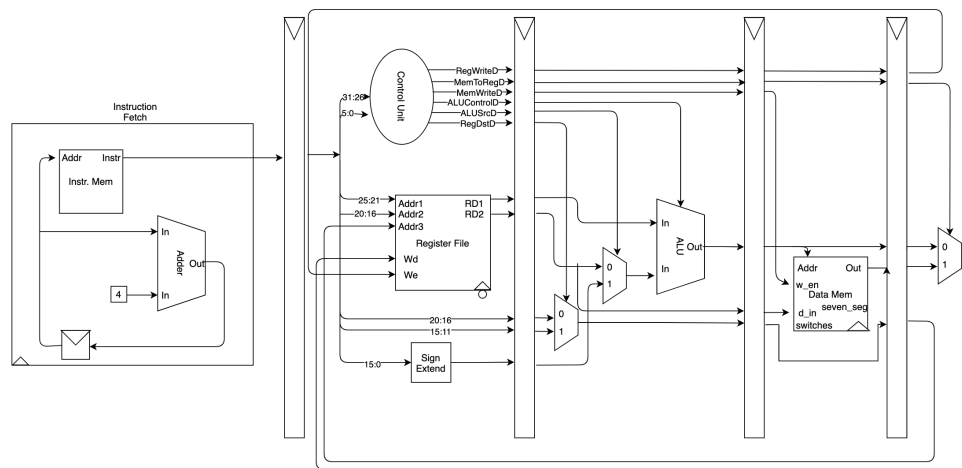


Figure 5.1: Pipelined MIPS Datapath

This exercise will focus on designing the final two stages in the Figure.

Pre-lab Activities

- Review the functionality of Verilog, as well as both the Memory and Writeback Stages
- Create a block diagram for the wrapper of the Memory Stage and Writeback Stage
- Explain the purpose and implementation difference between this stage and the register file stage.
- Print the sign-off sheet

Hardware Specification

Data Memory

This memory should be very similar to the Instruction Memory used in the Instruction Fetch stage. The values in the memory should be initialized, but this memory should be writable.

This memory has a significant difference, however, Address 1022 maps to the switches on the board and should not be writable, and address 1023 maps to the seven segment display and should not be read from. The reasons for this will be explained in the Memory Stage section under “Memory Modification Rationale”.

- Generics:
 - WIDTH: The width of each section of memory. Default to 32
 - ADDR_SPACE: Number of bits available for addressing. Default to 10.
- Inputs:
 - clk: Clock for the memory
 - w_en: Write Enable for writing to Memory
 - addr (ADDR_SPACE bits): Address to write data to and read data from
 - d_in (WIDTH bits): Data to be written to Memory
 - switches (16 bits): A vector of the switches on the board
- Outputs:
 - d_out (WIDTH bits): Data being read from addr
 - seven_seg (16 bits): A vector which outputs to a seven segment encoding module

Data Memory Verilog

```
module data_memory (CLK, W_EN, ADDR, D_IN, D_OUT, SWITCHES, SEVEN_SEG);
  parameter WIDTH = 32;
  parameter ADDR_SPACE = 10;

  input CLK, W_EN; // clock and write enable
  input [ADDR_SPACE-1:0] ADDR; // Address from ALU
  input [WIDTH-1:0] D_IN; // Write Data from Register File
  input [15:0] SWITCHES; // Switch input
  output reg [WIDTH-1:0] D_OUT; // Read Data to Writeback Mux
  output reg [15:0] SEVEN_SEG; // Seven Segment Display Output

  reg [WIDTH-1:0] mips_mem [(2**ADDR_SPACE)-1:0];

  always @(posedge CLK)
    if (W_EN)
      mips_mem[ADDR] <= D_IN;

  always @(posedge CLK)
    if (ADDR == 10'h3ff)
      if (W_EN)
        SEVEN_SEG <= D_IN[15:0];

  always @(posedge CLK)
    if (ADDR == 10'h3fe)
      D_OUT = {16'h0000, SWITCHES};
    else
```

```

        D_OUT = mips_mem[ADDR];

endmodule

```

Memory Stage

The total Memory Stage for the MIPS Datapath. This should contain an instance of the Data Memory, written in VHDL, and use several signals from previous stages to act on the memory.

- Inputs:
 - `clk`: System Clock
 - `rst`: Active-high reset for the seven-segment display controller
 - `RegWrite`: Determines if a register is being written to in the instruction
 - `MemtoReg`: Determines if memory is being read by the instruction
 - `WriteReg` (5 bits): The address of the register being written back to
 - `MemWrite`: Determines if memory is being written to
 - `ALUResult`: Result of the ALU from the Execute Stage
 - `WriteData`: Data to be written to the memory
 - `Switches` (16 bits): Switch input to the memory
- Outputs:
 - `RegWriteOut`: Control bit passthrough
 - `MemtoRegOut`: Control bit passthrough
 - `WriteRegOut` (5 bits): Control bits passthrough
 - `MemOut` (32 bits): Data retrieved from Data Memory
 - `ALUResultOut` (32 bits): ALU Result passthrough
 - `Active_Digit` (4 bits): Which digit on the seven segment display to output to
 - `Seven_Seg_Digit` (7 bits): Hex value output for seven segment display

Seven-Segment Display Usage: In order to convert from the hex values being output by the Data Memory to the values for the seven segment display, a seven segment converter has been provided on the MyCourses. It takes a 16 bit input and outputs which segment digit to set the LEDs for and what those LEDs should be set to. The seven segment display works by setting each digit repeatedly, and cycling through them fast enough that the human eye can't perceive the flickering. To learn more about how this works and how the controller was made, you can read this documentation: <https://www.fpga4student.com/2017/09/seven-segment-led-display-controller-basys3-fpga.html>.

To use the controller, simply map its `Active_Digit` and `Seven_Seg_Digit` outputs to the outputs of the same name on the Memory Stage. The constraints file will handle how it actually maps to the board.

Memory Modification Rationale: The additional connections to the switches and seven-segment display are for the purposes of being able to test the completed MIPS datapath on the Basys3 FPGA.

We make Address 1022 map to the switches, which allows the switches to be used as an input to the system. We make Address 1023 map to the seven-segment display, which gives the system a human-readable output.

For instance, a series of instructions could request the data at memory address 1022, add 5 to that, and store it back to address 1023.

Note: As a result of how this memory is implemented in an FPGA, the switches and seven-segment display do not actually interact with the rest of the memory. Writing a value to 1023 does not store that value anywhere except for the seven segment controller. Address 1022, as such, also cannot be written to, only read from.

Writeback Stage

- Inputs:
 - RegWrite: Register File write enable.
 - MemtoReg: Select for register write - ALUResult or ReadData.
 - ALUResult (32 bits): ALU Result from Execute Stage.
 - ReadData (32 bits): Data read from Memory.
 - WriteReg (5 bits): Register File write address.
- Outputs:
 - RegWriteOut: Register File write enable output.
 - WriteRegOut (5 bits): Register File write address output.
 - Result (32 bits): Data to write into the Register File.

Writeback Stage Verilog

```
module WriteBack (WriteReg, RegWrite, MemtoReg, ALUResult, ReadData,
    ↪ Result, WriteRegOut, RegWriteOut);

input [4:0] WriteReg;
input RegWrite, MemtoReg;
input [31:0] ALUResult, ReadData;
output reg [31:0] Result;
output [4:0] WriteRegOut;
output RegWriteOut;

always @(ReadData or MemtoReg or ALUResult)
begin
    if (MemtoReg)
        Result = ReadData;
    else
        Result = ALUResult;
end
```

```

assign WriteRegOut = WriteReg;
assign RegWriteOut  = RegWrite;

endmodule

```

Testbench

Verilog testbenches for both the Data Memory and the Writeback Stage are provided below. A VHDL testbench must be written which tests the same cases, as well as 4 additional cases per testbench which demonstrate proper functionality.

Data Memory Verilog Testbench

```

`timescale 1ns/1ps

module data_memory_tb();

    parameter WIDTH = 32;
    parameter ADDR_SPACE = 10;

    reg CLK = 0, W_EN; // clock and write enable
    reg [ADDR_SPACE-1:0] ADDR; // Address from ALU
    reg [WIDTH-1:0] D_IN; // Write Data from Register File
    reg [15:0] SWITCHES; // Switch input
    wire [WIDTH-1:0] D_OUT; // Read Data to Writeback Mux
    wire [15:0] SEVEN_SEG; // Seven Segment Display Output

    data_memory #(WIDTH,ADDR_SPACE) data_memory0 (
        .CLK(CLK), .W_EN(W_EN), .ADDR(ADDR), .D_IN(D_IN), .SWITCHES(SWITCHES),
        ↪ .D_OUT(D_OUT), .SEVEN_SEG(SEVEN_SEG));

    always
        #20 CLK = !CLK;

    initial begin
        @(negedge CLK) // write AAAA5555 to 1B
        W_EN = 1;
        ADDR = 10'h1B;
        D_IN = 32'hAAAA5555;

        @(negedge CLK); // write 5555AAAA to 1C
        ADDR = 10'h1C;
        D_IN = 32'h5555AAAA;

        @(negedge CLK) // read from 1B
        W_EN = 0;
        ADDR = 10'h1B;
        @(posedge CLK)
        if(!(D_OUT==32'haaaa5555)) $display("addr=%h,read=%h but should be
            ↪ haaaa5555", ADDR,D_OUT);

        @(negedge CLK); // read from 1C
        ADDR = 10'h1C;
    end

```

```

@(posedge CLK)
if(!(D_OUT==32'h5555aaaa)) $display("addr=%h,read=%h but should be
    ↪ h5555aaaa", ADDR,D_OUT);

//test switches that are read from memory when ADDR=1022
@(negedge CLK)
W_EN = 0;
SWITCHES = 16'h1111;
ADDR = 10'd1022;
@(posedge CLK)
if(!(D_OUT==32'h00001111)) $display("addr=%h,read=%h but should be
    ↪ h00001111", ADDR,D_OUT);

//test seven segment that receive the contents of address 1023
@(negedge CLK)
W_EN = 1;
ADDR = 10'd1023;
D_IN = 16'h3333;
@(posedge CLK)
if(!(SEVEN_SEG==32'h00003333)) $display("addr=%h,read=%h but should be
    ↪ h00003333", ADDR,SEVEN_SEG);
end

endmodule

```

Writeback Stage Verilog Testbench

```

`timescale 1 ns/1 ps
module WriteBackTB;
reg [4:0] WriteReg;
reg RegWrite, MemtoReg;
reg [31:0] ALUResult, ReadData;
wire [31:0] Result;
wire [4:0] WriteRegOut;
wire RegWriteOut;

WriteBack uut (.WriteReg(WriteReg),
               .RegWrite(RegWrite),
               .MemtoReg(MemtoReg),
               .ALUResult(ALUResult),
               .ReadData(ReadData),
               .Result(Result),
               .WriteRegOut(WriteRegOut),
               .RegWriteOut(RegWriteOut));

initial begin
    WriteReg = 5'b00000; #20
    if (WriteRegOut != 5'b00000) $display("WriteRegOut FAILED in %m");
    WriteReg = 5'b11111; #20
    if (WriteRegOut != 5'b11111) $display("WriteRegOut FAILED in %m");

    RegWrite = 1'b0; #20
    if (RegWriteOut != 1'b0) $display("RegWriteOut FAILED in %m");
    RegWrite = 1'b1; #20

```

```

if (RegWriteOut != 1'b1) $display("RegWriteOut FAILED in %m");

MemtoReg = 1'b0;
ALUResult = 32'h0;
ReadData = 32'hFFFFFFF; #20
if (Result != 32'h0) $display("Result FAILED with reg = 0 in %m");
MemtoReg = 1'b1; #20
if (Result != 32'hFFFFFFF) $display("Result FAILED with mem = FFFFFFFF
    ↪ in %m");

MemtoReg = 1'b0;
ALUResult = 32'h55555555;
ReadData = 32'hAAAAAAAA; #20
if (Result != 32'h55555555) $display("Result FAILED with reg = 55555555
    ↪ in %m");
MemtoReg = 1'b1; #20
if (Result != 32'hAAAAAAAA) $display("Result FAILED in mem = AAAAAAAA %
    ↪ m");

end

endmodule

```

Hardware Demonstration

1. Download and review the Seven Segment display code on MyCourses.
2. Place the value x"0f1e2d3c" into the memory at address 1023, so that it would display on the Seven-Segment display.
3. Modify the constraints file to connect any remaining signals which are not already mapped so that they do not interfere with the signals already present.
4. Remove unnecessary inputs and outputs for this test, to make the design small enough to fit on the FPGA.
 - The only I/O that should be left are as follows:
 - clk
 - rst
 - Switches
 - MemOut
 - Active_Digit
 - Seven_Seg_Digit
5. The hardware design should function as follows:
 - The switches should be used as input to the system when reading address 1022 instead of reading from memory.
 - The seven-segment display should display the value stored in memory at address 1023.
6. Implement the design on the Basys3 and show that the correct value appears on the display.

Procedure

Part 1 - Memory Stage

1. Create a new Project in Xilinx Vivado for the Memory Stage.
2. Create a new VHDL file for the Data Memory.
3. Analyze the provided Verilog for the Data Memory.
4. Translate the provided Verilog for the Data Memory into properly commented and formatted VHDL, making sure to keep the same functionality.
5. Create a TB for the Data Memory by translating the provided Verilog for the Data Memory testbench into VHDL, making sure to keep the same functionality.
6. Create a new VHDL file for the Memory Stage.
7. Build the full Memory Stage using the Data Memory as a component.
8. Write a testbench which properly tests the stage, this testbench must use assert statements to verify the functionality.
9. Test the full Memory Stage with a Behavioral Simulation.
10. Synthesize the full Memory Stage and test it using a Post-Synthesis Timing Simulation.
11. Implement the full Memory Stage and test it using a Post-Implementation Timing Simulation.
12. Demonstrate the above steps to a TA or Lab Instructor.
13. Screenshot the appropriate waveforms for the report, making sure they are readable.

Part 2 - Writeback Stage

1. Create a new VHDL file for the Writeback Stage.
2. Analyze the provided Verilog for the Writeback Stage.
3. Translate the provided Verilog for the Writeback stage into properly commented and formatted VHDL, making sure to keep the same functionality.
4. Create a TB for the Writeback Stage by translating the provided Verilog for the Writeback stage testbench, making sure to keep the same functionality.
5. Test the Writeback stage with a Behavioral Simulation.
6. Synthesize the Writeback stage and test it using a Post-Synthesis Timing Simulation.
7. Implement the Writeback stage and test it using a Post-Implementation Timing Simulation.
8. Demonstrate the above steps to a TA or Lab Instructor.
9. Screenshot the appropriate waveforms for the report, making sure they are readable.
10. Submit the source code and report online to the appropriate MyCourses dropbox.

Exercise 5: Memory & Writeback Stage

Student's Name: _____

Section: _____

PreLab		Point Value	Points Earned	Comments
PreLab	Memory Stage Wrapper	2		
	Discussion of Memory and write back stage use in the larger MIPS Design	5		
	Writeback Stage Wrapper	2		

Demo		Point Value	Points Earned	Date
Part 1 – Memory Stage	Behavioral Simulation	15		
	Post-Synthesis Timing Simulation	5		
	Explain the purpose and implementation difference between this stage and the register file stage.	5		
	Post-Implementation Timing Simulation	5		
	Hardware Demonstration	6		
Part 2 – WB Stage	Behavioral Simulation	15		
	Vivado Schematic	5		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

Exercise 5: Memory & Writeback Stage

Report		Point Value	Points Earned	Comments
Abstract		5		
Design Methodology	Discussion of Verilog Functionality for Memory	5		
	Discussion of Verilog Functionality for Writeback	5		
Results & Analysis	Behavioral Simulation for Memory	5		
	Behavioral Simulation for Writeback	5		
	Discussion of Results	5		
Conclusion		5		
Source Code (Style)		5		
Total for prelab, demo, and report		100		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

In-Lab Deliverables

Part 1: Memory Stage

- VHDL Code for Memory Stage.
- VHDL Code for Memory Stage Testbench.
- VHDL Code for Instruction Memory.
- VHDL Code for Instruction Memory Testbench.

Part 2: Writeback Stage

- VHDL Code for Writeback Stage.
- VHDL Code for Writeback Stage Testbench.

Lab Report Deliverables

Cover Page

- Proper cover page that includes exercise number and name as well as student's name, lab section, lab instructor, teaching assistants, and lecture instructor.

Abstract

- Objective of the exercise.
- Summary of what was done.
- How the design was tested.
- Results of exercise (successful or unsuccessful).

Design Methodology

- Analysis of the functionality of the memory stage and writeback stage.
 - What was the purpose of each module?
 - How was the module implemented?

Results and Analysis

- Waveforms and discussion of:
 - Behavioral simulation of the Memory stage.
 - Post implementation simulation of the Memory Stage.
 - Behavioral simulation of the Writeback stage.

Cite specific examples of proof that the waveforms are correct. Stating that the wave matched the expected results with no test cases cited does not explain what the expected results were, or how they matched the testbench.

- Discussion of test cases.

Conclusion

- Restate objective and summary of exercise.
- State the final results of the exercise.
- Reflections of lessons learned and discussion of future use of what was learned.

Other

- Report should be original and follow digital system design report guidelines.

Other Deliverables

- All VHDL code must be properly commented and have proper comment headers that detail the student's name, lab section, and file description.
- All submissions must follow the submission guidelines detailed within the preliminary matter of this manual.
- **ALL VHDL CODE FOR BOTH PARTS 1 AND 2 MUST BE SUBMITTED TO MYCOURSES AS .TXT FILES TO RECEIVE A GRADE.**

Exercise 6 Pipelined MIPS Processor

Introduction

Students will now assemble the 5 individual stages of the microprocessor to create the overall microprocessor architecture as shown in Figure 6.1. In order to do this outputs of each stage need to be captured into a register. The output of this register needs to be fed as inputs to the next stage. All the registers will operate with the same clock signal to maintain a pipelined operation of the microprocessor.

NOTE: for the out output of this lab, we want to see the highlighted signal on your waveform.

Working stages from Exercises 3, 4, and 5 are needed for this exercise.

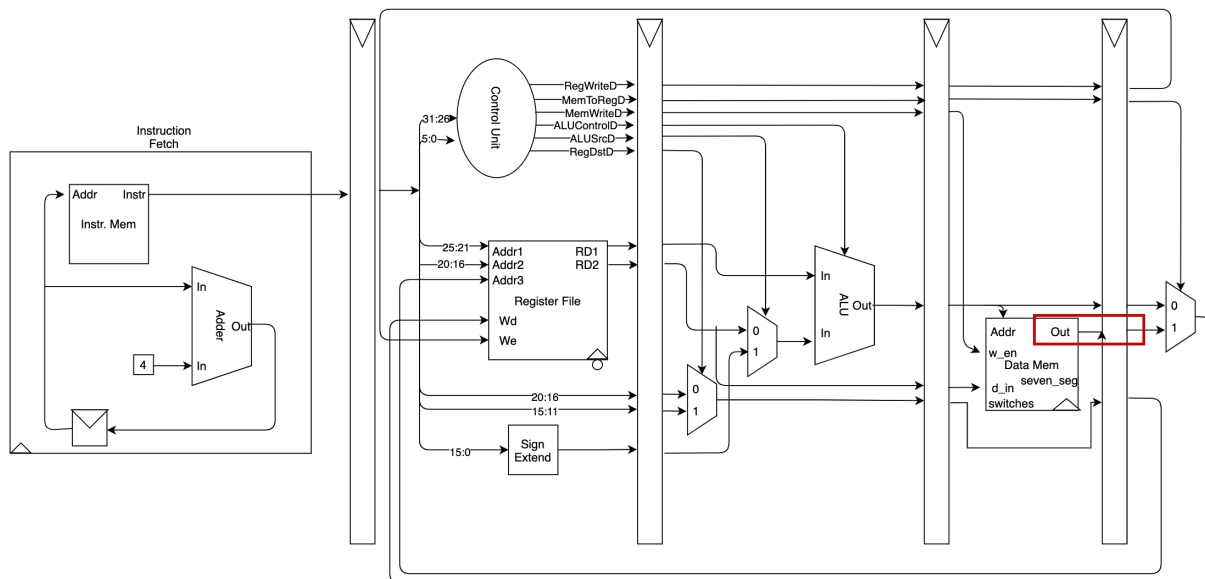


Figure 6.1: Pipelined MIPS Processor, with highlight



Ignore any signals dealing with branching — these will not be implemented in this version of the MIPS processor.



It is not necessary to instantiate all of the registers for pipelining. This can be done with a single clocked process.

Pre-Lab

- By this point in time you should have many pieces of MIPS that you will now attach together via registers. You have probably asking yourself, “why not just do it as one solid block with latches instead of registers why even bother having a clock, and pipeline.” Do your own research

on computer architecture, and answer the question “what is the advantage of pipe-lining in computer architecture”

- Write the pseudo-code for creating a Fibonacci series, then write the subsequent MIPS Assembly code, then translate that to machine code, in the form of hex. Remember that there are no branches or jumps in this implementation of the MIPS processor. If you need help, near the bottom of this section is several resources that can help you convert Assembly MIPS to machine code.
- Create a block diagram for the completed MIPS datapath. The stages should be shown as black boxes with the correct ports. The block diagram must include the following:
 - Registers between stages.
 - All signal names for connecting the stages and registers.
 - Bit widths for all signals.

Procedure

Part 1

1. Create an upper level wrapper (a structural higher level) that instantiates all the 5 stages and the registers between them. This top-level entity is the micro-processor.
 - A. The inputs of the upper level shell will be a clock and reset signal.
 - B. The outputs of the microprocessor will be the Write data from the Memory stage for the seven-segment display as well as the ALU result from the Execute stage. It is highlighted in the figure figure 6.1.
 - C. The purpose for the ALU result being pulled out is so Vivado does not interpret the system as a closed loop and optimize the design away.
 - D. Every stage will need to be instantiated and connected to the appropriate registers to ensure proper timing between stages.



Signals must be created for every Input/Output for the 5 stages of the MIPS Processor. These signals will be updated in clocked processes (registers) so they update at the same time.

2. Populate the instruction memory with instructions that test the operation of the all the various instructions as listed in Lab Exercise 3 (ID Stage). Every instruction must be tested at least once.
3. Create a testbench such that the different kinds of instructions can be fetched from the instruction memory clock cycle by clock cycle and execute them. The testbench must be able to show the output of the ALU and the data written back or read/written to the data memory (output of the Mem stage).

4. Demonstrate the program on the Basys-3 Board. Do so by writing a program which reads the values from the switches, performs two different ALU operations on that value, and writes the result out to the seven-segment display.



In order to get the design to properly implement on the board, a clock wizard will likely need to be created to slow the clock. A tutorial for this is included in Appendix G of the main Manual.

Part 2

1. Create instructions based on the pseudo-code written in the Prelab section to generate at least the first 10 Fibonacci Numbers using the created MIPS processor. The numbers should be written back in the data memory. Show the output of the ALU and the content of the data memory in the behavioral to demonstrate the creation of the Fibonacci series.
2. Demonstrate this same functionality on the Basys-3 board by writing the 10 Fibonacci numbers to the seven-segment display. Because the Clock is very fast, it is required that one introduce delay in the system to make it actually see the output.



To debug, add intermediate signals to the wave window. Add dividers and group them by stage. This way, all of the data can be traced through the entire pipeline.

Converting from MIPS to machine code

Follow this link to go to an awesome converter, that will quickly, and precisely turn your MIPS assembly to machine code.

https://www.eg.bucknell.edu/~csci320/mips_web/

To also aid in this attached in MyCourses you should find a file titled:

MIPS_MANUAL_CONVERSION_GUIDE.pdf. If it is not their politely inform your TA, and instructor.

Exercise 6: Pipelined MIPS Processor

Student's Name: _____

Section: _____

PreLab		Point Value	Points Earned	Comments
PreLab	Fibonacci Pseudo-Code, MIPS CODE, & MACHINE HEX code	10		
	Talk on pipelining in processors	5		
	Block Diagram	8		

Demo		Point Value	Points Earned	Date
Part 1	Behavioral Testbench	5		
	Hardware Demo	10		
Part 2	Behavioral Testbench	5		
	Hardware Demo	11		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

Exercise 6: Pipelined MIPS Processor

Report		Point Value	Points Earned	Comments
Abstract		5		
Design Methodology	Discussion of Full MIPS Pipeline Functionality	5		
	Discussion for Method for Calculating Fibonacci Numbers	4		
Results & Analysis	Part A: Behavioral Waveform	4		
	Discussion of the advantages and disadvantages of pipe lining	10		
	Part A: Result Discussion	4		
	Part B: Behavioral Waveform	4		
	Part B: Result Discussion	4		
Conclusion		5		
Source Code (Style)		5		
Total for prelab, demo, and report		100		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

In-Lab Deliverables

Part 1 — Pipelined MIPS Processor

Working stages from Exercises 3, 4, and 5 are needed for this exercise.

- VHDL Code for Pipelines MIPS.
 - Design implements specifications listed in the Hardware Specification section of this exercise.
 - Design is structural and uses the previously created stages to create a pipelined MIPS processor.
 - Instructions for program are loaded into Instruction Memory.
 - Results for instructions are saved in memory.
- VHDL Testbench Code for Pipelined MIPS.
 - Testbench **does not** use a record array to contain test cases. Test cases are to be stimulated with the Instruction Memory.
 - Testbench tests each instruction at least once.
- VHDL Testbench must successfully perform behavioral simulations.
- Hardware Demonstration.

Part 2 — Control Unit and Instruction Decode

Working stages from Exercises 3, 4, and 5 are needed for this exercise.

- VHDL Code for Fibonacci Sequence in Instruction Memory.
 - Fibonacci Code is written in Instruction Memory.
 - Fibonacci Code writes the first ten Fibonacci numbers into memory.
- VHDL Testbench must successfully perform behavioral simulations.
- Hardware Demonstration.

Lab Report Deliverables

Cover Page

- Proper cover page that includes exercise number and name as well as student's name, lab section, lab instructor, teaching assistants, and lecture instructor.

Abstract

- Objective of the exercise.
- Summary of what was done.
- How the design was tested.
- Results of exercise (successful or unsuccessful).

Design Methodology

- Discussion of the Full MIPS Pipeline and its functionality. Discussion should include a brief explanation of what each stage does and how they connect to each other to form a pipeline.
- Discussion of Methodology for Calculating the Fibonacci Numbers. Discussion should include a brief explanation of how the Fibonacci numbers are calculated and how the code is written in Instruction Memory.

Results and Analysis

- Waveforms and explanations demonstrating each instruction.
- Waveforms and explanations demonstrating the Fibonacci sequence.

Conclusion

- Restate objective and summary of exercise.
- State the final results of the exercise.
- Reflections of lessons learned and discussion of future use of what was learned.

Other

- Report should be original and follow digital system design report guidelines.

Other Deliverables

- All VHDL code must be properly commented and have proper comment headers that detail the student's name, lab section, and file description.
- All submissions must follow the submission guidelines detailed within the preliminary matter of this manual.
- **ALL VHDL CODE FOR BOTH PARTS 1 AND 2 MUST BE SUBMITTED TO MYCOURSES AS .TXT FILES TO RECEIVE A GRADE.**

Exercise 7 Processor Timing (Extra Credit)

Introduction

In this exercise, students will observe and understand timing concepts in a pipelined microprocessor. The objective of this exercise is to enable students to verify static timing performance of the designed 5-stage microprocessor. **This Exercise is Extra Credit. The full Exercise and Report must be completed to obtain extra credit points.**

Procedure

1. Use the assembled 5-stage MIPS based microprocessor to verify if the architecture can be operated at a given clock frequency. Use the clocking wizard IP Core tutorial in Appendix H to generate clock frequencies within a specified range.
2. Starting at 10MHz, synthesize and implement the design and check if setup and hold times are satisfied by analyzing the Timing Reports. Record the clock frequency, setup time, hold time, and if the processor succeeded or failed.
3. Repeat step 2 in steps of 5MHz until the design fails (setup or hold times are violated).
4. When the design fails in step 3, repeat step 2 in steps of 1MHz until the design fails.
5. Record the highest clock frequency at which the design can be operated without violating setup or hold times.

Questions

Answer the following questions:

- What are setup and hold times?
- How would violating setup and hold times can cause a design to fail?

Exercise 7: Processor Timing

Student's Name: _____ Section: _____

Questions		Point Value	Points Earned	Comments
Questions	Definitions of Setup and Hold Times	10		

Demo		Point Value	Points Earned	Date
Demo	Post Implementation Simulation of MIPS Datapath	25		
	Evaluation of Setup and Hold Times	40		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.

This Exercise is Extra Credit. The full Exercise and Report must be completed to obtain extra credit points.

Exercise 7: Processor Timing

Report		Point Value	Points Earned	Comments
Answer to Questions		5		
Results & Analysis	Post-Implementation Timing	5		
	Table of Results	10		
Conclusion		5		
Total for questions, demo, and report		100		

To receive any grading credit students must earn points for both the demonstration and the report. Students must also submit all VHDL code as .TXT files. Failing to do so may result in no credit for the assignment.



The “Report” for this part of the project is not a full report. Please only include the things highlighted in the rubric above. Format the table however you see fit to show the appropriate information.



The “Report” should still be semi-formal and proper analysis of results is still required for full points.

This Exercise is Extra Credit. The full Exercise and Report must be completed to obtain extra credit points.

This Exercise is Extra Credit. The full Exercise and Report must be completed to obtain extra credit points.

In-Lab Deliverables

Processor Timing

A working MIPS Pipeline is needed for this exercise.

- MIPS Pipeline must successfully perform Post-Implementation Timing simulation.
- Students must be able to demonstrate the maximum clock frequency the design can run at.
- Students must show recorded data in a table.

Lab Report Deliverables

- A compiled documentation of the following:
 - Answers to prelab questions.
 - A table of the recorded clock frequencies. The table should include the recorded clock frequency, setup time, hold time, and if the processor succeeded or failed.
 - Implementation waveforms for each of the frequencies, noting which ones failed.
 - Note that simply showing the waveforms is not sufficient for this exercise. Students must be able to analyze the waveforms and explain why the design failed at certain frequencies.

Other Deliverables

- All VHDL code must be properly commented and have proper comment headers that detail the student's name, lab section, and file description.
- All submissions must follow the submission guidelines detailed within the preliminary matter of this manual.
- **ALL VHDL CODE FOR THIS EXERCISE BE SUBMITTED TO MYCOURSES AS .TXT FILES TO RECEIVE A GRADE.**

This Exercise is Extra Credit. The full Exercise and Report must be completed to obtain extra credit points.

Appendix A Xilinx Vivado Tutorial

Objective

The following is a tutorial on how to create and test a simple VHDL design in the Xilinx Vivado Design Suite. Vivado is a comprehensive software suite for FPGA development. It includes a source code editor, synthesis tools, implementation tools, a simulator, and tools for loading a design onto FPGAs.

Vivado WebPack edition is available for students, free of charge, as explained here.

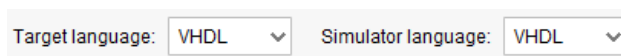
- If you install Vivado to your personal computer, you will likely not be able to select the board that we will be using for these labs. Follow the instructions here to add them: <https://reference.digilentinc.com/reference/software/vivado/board-files>

Design

Here, we walk through the process of creating a Vivado project and design files. For this example, we will create a dataflow design for an 4-bit partial arithmetic logic unit (ALU) capable of NOT and Logical Left Shift (SLL) operations.

1. Project Creation Wizard

- (a) Create a new project from either the “Quick Start” or “File” menu. This will launch the project creation wizard. Click Next to get started.
- (b) **Project Name:** Give the project a name of your choice and select a location with fast storage (local disk is recommended). Click Next.
- (c) **Project Type:** Select ”RTL Project.” Click Next.
- (d) **Add Sources:**
 - (i) At the bottom of the window, select VHDL as your target language and simulator language.



Target language: VHDL Simulator language: VHDL

- (ii) For each of the following components, click **Create File**, specify the file name, and click OK.
 - (1) notN.vhd
 - (2) sllN.vhd
 - (3) alu4.vhd
 - (4) global_pkg.vhd
 - (iii) Click Next to continue
- (e) **Add Constraints:** Skip this step for now. Click Next.
 - (f) **Default Part:** Click Board, then select Basys3, then click Next.
 - (g) Click Finish to complete the project creation wizard.

2. **Define Modules:** This will pop-up when your project opens. It provides a GUI to automatically generate entity declarations. We want to do this ourselves, so skip it by clicking OK. If you get a warning about unchanged files, click Yes to ignore it.
3. In the **Sources** panel, double-click on **notN.vhd** to open it. Type the source code from Appendix B and save. Repeat this process for **sl1N.vhd**, **globals.vhd** and **alu4.vhd** with Appendix C, D and E respectively. At the end of this process, your **Sources** panel should look like this:



Copying from the pdf file can cause spacing issues and hidden characters. If you run into problems that seemingly don't make sense, you may need to type out the file.

The tree now represents your hierarchical/structural design. This is helpful in understanding the architecture of a design at a glance. However, if you ever want to see a flat list of files in the project, click **Compile Order** along the bottom of the **Sources** panel. See Appendix I (Common Problems) if any of the files are unreferenced.

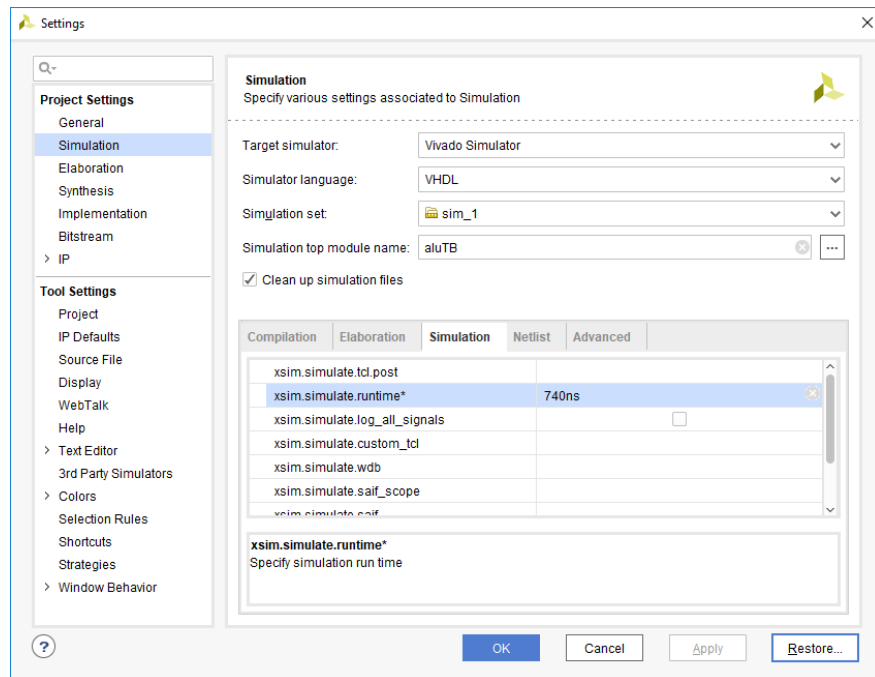
Behavioral Simulation

A crucial step in the design process is testing whether or not your design provides the expected functionality. The simplest way to do this is by creating a VHDL test bench and running a behavioral simulation. This is an ideal simulation and does not account for any actual hardware. However, it is the fastest type of simulation to run and the results are easy to read and parse.

1. Creating a Simulation Source

- (a) From the **Sources** panel, click . This will launch the **Add Sources** wizard once again.
- (b) Choose **Add or create simulation sources**. Click Next.
- (c) Click **Create File**. Name it **aluTB** and click **OK**. Then, from the wizard, click Finish.

2. **Define Module:** Once again, this wizard will pop up. Again, we can skip it by clicking OK and then Yes.
3. From the sources panel, under **Simulation Sources** > **sim_1**, double-click on **aluTB.vhd** to open it. Copy the source code from Appendix F and save your file.
4. To specify the length of the simulation, go to the Flow Navigator and right click on **SIMULATION**. Click “Simulation Settings...”. Under the **Simulation** tab, set the value of **xsim.simulate.runtime** to **740ns**, like so:



Click **Apply**, then click **OK**.

5. From the Flow Navigator, under **SIMULATION**, click **Run Simulation**. Then click **Run Behavioral Simulation**.
6. Vivado should show a waveform of the simulation to to 200ns. Behavioral simulations are logical only. As such, no design is synthesized and no hardware delays are incurred. Check to make sure that all combinations of operations and operands produce correct results.

Synthesis & Post-Synthesis Simulation

Synthesis “compiles” your design into a gate-level netlist, represented by the UNISIM library, a Xilinx library containing basic primitives. In this step, we will synthesize ALU design, check the synthesis reports, view the register transfer level (RTL) diagram, and run a Post-Synthesis simulation.

1. From the **Flow Navigator**, under **SYNTHESIS**, click **Run Synthesis**, then click **OK**. The status indicator in the top-right of your screen should indicate that this process is running. This could take a few minutes. Wait until the status indicator displays **Synthesis Complete**.

2. To simulate this model, go to **Flow Navigator > SIMULATION > Run Simulation > Run Post-Synthesis Timing Simulation**. If given a warning, click **Yes**. The waveforms in this simulation should include delays incurred by the gates in the model. As such, there should be setup times for some signals and signals may change value multiple times before becoming stable.
3. One debugging technique is to examine the RTL diagram of a design. To do this, under **Flow Navigator > SYNTHESIS > Open Synthesized Design**, click **Schematic**. The RTL schematic can be a useful debugging tool, as it shows the connections between internal components of the synthesized design.
4. One can determine the resource utilization under **SYNTHESIS > Report Utilization**. Determine how many Slice LUTs are used in this design.

Errors in this type of simulation stage are often (but not always) the result of a latch or some other design flaw within a process.

NOTE: A functional simulation also tests the gate-level model, but does not account for delays. The upside is that, due to predictable timing, it is easier to automate testing.

Implementation & Post-Implementation Simulation

Implementation places and routes your synthesized design into a model of the FPGA, allowing you to simulate how your design will behave on the actual hardware.

1. From the **Flow Navigator**, under **IMPLEMENTATION**, click **Run Implementation**, then click **OK**. The status indicator in the top-right of your screen should indicate that this process is running. This could take a few minutes. Wait until the status indicator displays **Implementation Complete**.
2. As with Synthesis, Implementation also generates a schematic. View it under **Flow Navigator > IMPLEMENTATION > Open Implemented Design > Schematic**.
3. To view the resource utilization of the implemented model, go to **Flow Navigator > IMPLEMENTATION > Open Implemented Design > Report Utilization**. Note the number of slice LUTs being used. Also note the number and types of slices being used.
4. To simulate this model, go to **Flow Navigator > SIMULATION > Run Simulation > Run Post-Implementation Timing Simulation**. If given a warning, click **Yes**. The waveforms in this simulation should include delays incurred by the gates in the model. As such, there should be setup times for some signals and signals may change value multiple times before becoming stable.

Again, errors in this type of simulation stage are often (but not always) the result of a latch or some other design flaw within a process.

NOTE: A functional simulation also tests the implemented model, but does not account for delays. The upside is that, due to predictable timing, it is easier to automate testing.

RTL Analysis

Debugging can be the most time consuming step in the design process. To reduce this time, Xilinx offers several tools like stepping and schematics. This section covers schematics using the shift left file.

1. Interpret the sllN.vhd code's generate command to complete the table below.

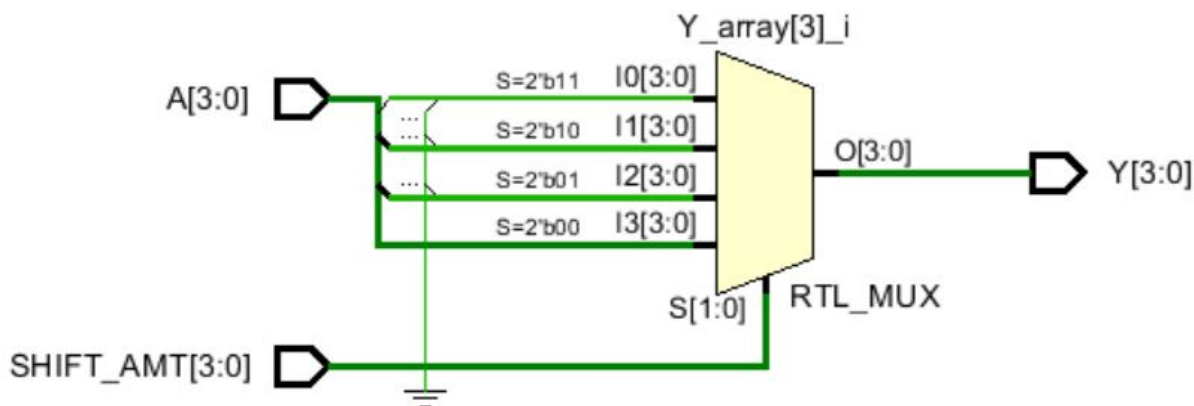
S	Y(3)	Y(2)	Y(1)	Y(0)	assignment
"11"					$Y(3) \leq A(0)$ and $Y(2 \text{ downto } 0) \leq '0'$
"10"					
"01"	A(2)	A(1)	A(0)	'0'	
"00"	A(3)	A(2)	A(1)	A(0)	$Y \leq A$

2. Create a new Project for Shift Left

- (a) Copy sllN.vhd to a separate, new folder.
- (b) Close the current project from the "File" menu.
- (c) Create a new project as you did before **in the new sllN directory that includes only sllN.vhd.**

3. Run Synthesis

4. From the Flow Navigator, under **RTL ANALYSIS**, click **Schematic** and **OK** to generate the figure below.



5. Hold the mouse over the second set of ellipse in row $S=2'b10$ of the schematic to display $1:0==3:2$. This is telling you that $A(1 \text{ downto } 0)$ is connected to $Y(3 \text{ downto } 2)$. The ground line tells you that the rest of the signals, $Y(1 \text{ downto } 0)$, are connected to ground. This is a shift of 2 to the left.
6. Complete the table for a shift right algorithm below.

S	Y(3)	Y(2)	Y(1)	Y(0)	assignment
"11"	'0'	'0'	'0'	A(3)	
"10"					Y(3 downto 2) <= '0' and Y(1 downto 0) <= A(3 downto 2)
"01"					
"00"	A(3)	A(2)	A(1)	A(0)	Y <= A

7. Create a new Project for Shift Right

- Copy srlN.vhd and rename it srlN.vhd. Create a separate, new folder, for srlN.vhd.
- Close the current project from the "File" menu.
- Create a new project as you did before **in the new srlN directory that includes only srlN.vhd**. Change the code so it shifts right as shown in the table above. A key component to this method is keeping N small. A small N keeps the schematic view manageable and easy to interpret. Once N=4 is verified, N=32 is likely to work nicely.

8. Run Synthesis

- From the Flow Navigator, under **RTL ANALYSIS**, click **Schematic** and **OK** to generate a schematic view.

Hardware

In this step, we will load the partial ALU design onto a Basys3 FPGA development kit. To do this, we will generate a "bitstream" file, which contains the necessary information for programming the target device with the design.

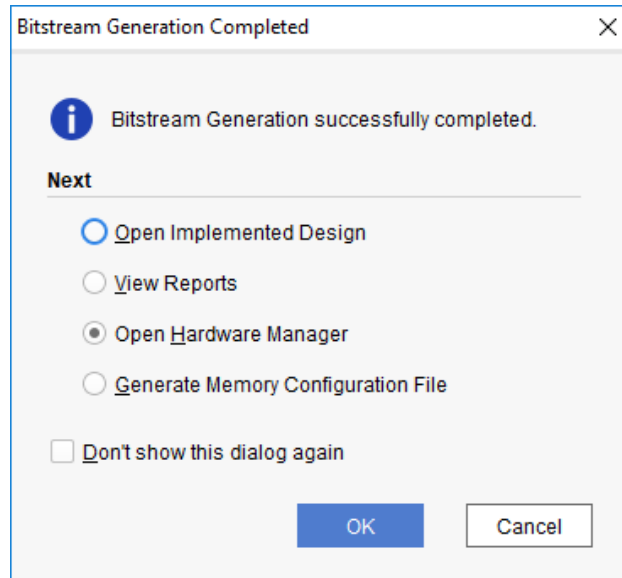
1. Create Constraints

- In order to map ports on the top-level design to pins on the FPGA, a Xilinx Design Constraints (XDC) file is required. To create one, click  in the **Sources** panel.
- Click **Add or create constraints**, then click **Next**.
- Click **Create File**, enter "adc4" for the **File name**, click **OK**, then click **Finish**.
- Copy the contents of Appendix G to adc4.xdc. This will map SW3 through SW0 to Port B, SW7 through SW4 to Port A, and SW15 to Port OP.
Note: When copying do not copy from a pdf viewer in a web browser. It will likely add hidden characters that will cause errors. Download the manual and copy from there, or type it out by hand.
- Save the file
- To create constraint files for future exercises use the master constraint file here https://github.com/Digilent/Basys3/blob/master/Projects/XADC_Demo/src/constraints/Basys3_Master.xdc**

- To generate the bitstream file, go to the **Flow Navigator > PROGRAM AND DEBUG**, and then click **Generate Bitstream** and wait for that process to complete. You may be asked

to if you'd like to re-run the Synthesis and Implementation steps. If that dialog appears, hit **Yes**.

3. After some time, the following message box should pop up:



Click **OK** to open the **Hardware Manager**.

4. Now connect your Basys3 to your workstation using a microUSB cable.
5. Flip the power switch SW16 up to turn on the device. Windows may take a few minutes to install drivers for the device.
6. Near the top of the screen, you should see a green bar with the link **Open target**. Click that link, then click **Auto Connect**.
7. The green bar should then show a link to **Program device**. Click it, then click **Program**.
8. At this point, the ALU design should be loaded onto the device. Test it by changing inputs using the aforementioned switches. The output of the ALU is displayed via the four rightmost LEDs.
9. Close **Hardware Manager**
10. Power off the Basys3 using SW16 and disconnect it from your workstation.

Appendix B notN Source Code

```
-----  
-- Company:  Rochester Institute of Technology (RIT)  
-- Engineer: <YOUR_NAME_HERE> (<YOUR_EMAIL_HERE>)  
--  
-- Create Date:    <CREATION_TIME_HERE>  
-- Design Name:    notN  
-- Module Name:    notN - dataflow  
-- Project Name:    <PROJECT_NAME_HERE>  
-- Target Devices: Basys3  
--  
-- Description: N-bit bitwise NOT unit  
-----  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity notN is  
    GENERIC (N : INTEGER := 4); --bit width  
    PORT (  
        A : IN std_logic_vector(N-1 downto 0);  
        Y : OUT std_logic_vector(N-1 downto 0)  
    );  
end notN;  
  
architecture dataflow of notN is  
begin  
    Y <= not A;  
end dataflow;
```


Appendix C sllN Source Code

```
-----
-- Company:   Rochester Institute of Technology (RIT)
-- Engineer:  <YOUR_NAME_HERE> (<YOUR_EMAIL_HERE>)
--
-- Create Date:    <CREATION_TIME_HERE>
-- Design Name:    sllN
-- Module Name:    sllN - behavioral
-- Project Name:   <PROJECT_NAME_HERE>
-- Target Devices: Basys3
--
-- Description: N-bit logical left shift (SLL) unit
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity sllN is
    GENERIC (N : INTEGER := 4;    --bit width
             M : INTEGER := 2);  --shift bits
    PORT (
        A          : IN std_logic_vector(N-1 downto 0);
        SHIFT_AMT   : IN std_logic_vector(M-1 downto 0);
        Y          : OUT std_logic_vector(N-1 downto 0)
    );
end sllN;

architecture behavioral of sllN is
    -- create array of vectors to hold each of n shifters
    type shifty_array is array(N-1 downto 0) of std_logic_vector(N-1
        ↪ downto 0);
    signal aSLL : shifty_array;

begin
    generateSLL: for i in 0 to N-1 generate
        aSLL(i)(N-1 downto i) <= A(N-1-i downto 0);
        left_fill: if i > 0 generate
            aSLL(i)(i-1 downto 0) <= (others => '0');
        end generate left_fill;
    end generate generateSLL;
    -- The value of shift_amt (in binary) determines number of bits A is
    ↪ shifted.
    -- Since shift_amt (in decimal) must not exceed n-1 so only M bits are
    ↪ used. The default or N=4,
    -- will require 2 shift bits (M=2), because 2^2 = 4, the maximum shift
    ↪ .
    -- In all cases, 2^M = N.
    Y <= aSLL(to_integer(unsigned(SHIFT_AMT)));
end behavioral;
```


Appendix D globals Package (library)

```
-----  
-- Company:  Rochester Institute of Technology (RIT)  
-- Engineer: <YOUR_NAME_HERE> (<YOUR_EMAIL_HERE>)  
--  
-- Create Date:    <CREATION_TIME_HERE>  
-- Design Name:    globals  
-- Module Name:    globals - package (library)  
-- Project Name:    <PROJECT_NAME_HERE>  
-- Target Devices: Basys3  
--  
-- Description: Constants used in top and test bench level  
--      Xilinx does not like generics in the top level of a design  
-----  
  
library ieee;  
use ieee.std_logic_1164.all;  
package globals is  
    constant N : INTEGER := 4;  
    constant M : INTEGER := 2;  
end;
```


Appendix E alu4 Source Code

```
-----
-- Company:  Rochester Institute of Technology (RIT)
-- Engineer: <YOUR_NAME_HERE> (<YOUR_EMAIL_HERE>)
--
-- Create Date:    <CREATION_TIME_HERE>
-- Design Name:    alu4
-- Module Name:    alu4 - structural
-- Project Name:   <PROJECT_NAME_HERE>
-- Target Devices: Basys3
--
-- Description: Partial 4-bit Arithmetic Logic Unit
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.globals.all; -- provides N and M to top level

entity alu4 is
    PORT (
        A : IN std_logic_vector(N-1 downto 0);
        B : IN std_logic_vector(N-1 downto 0);
        OP : IN std_logic;
        Y : OUT std_logic_vector(N-1 downto 0)
    );
end alu4;

architecture structural of alu4 is
-- inverter component declaration
    Component notN is
        GENERIC ( N : INTEGER := 4); -- bit width
        PORT (
            A : IN std_logic_vector(N-1 downto 0);
            Y : OUT std_logic_vector(N-1 downto 0)
        );
    end Component;

-- skip shift left component declaration, use entity work.
-- this is done so you can see code with and without components.

    signal not_result : std_logic_vector(3 downto 0);
    signal sll_result : std_logic_vector(3 downto 0);

begin
    -- Instantiate the inverter, using component
    not_comp: notN
        generic map ( N => N)
        port map ( A => A, Y => not_result );

    -- Instantiate the SLL unit, without component
    sll_comp: entity work.sllN
        generic map ( N => N, M => M)
```

```

    port map ( A=> A, SHIFT_AMT => B(M-1 downto 0), Y => sll_result );

    -- Use OP to control which operation to show/perform
    Y <= not_result when OP = '0' else -- NOT
        sll_result;                    -- SLL
end structural;

```


Appendix F aluTB Source Code

```
-----
-- Company:  Rochester Institute of Technology (RIT)
-- Engineer:  <YOUR_NAME_HERE> (<YOUR_EMAIL_HERE>)
--
-- Create Date:    <CREATION_TIME_HERE>
-- Design Name:    aluTB
-- Module Name:    aluTB - behavioral
-- Project Name:    <PROJECT_NAME_HERE>
--
-- Description: Testbec\nch for Partial 32-bit ALU
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.globals.all;

entity aluTB is
end aluTB;

architecture Behavioral of aluTB is
--Declare the ALU component
    Component alu4 is
        PORT (
            A    : IN std_logic_vector(N-1 downto 0);
            B    : IN std_logic_vector(N-1 downto 0);
            OP    : IN std_logic;
            Y     : OUT std_logic_vector(N-1 downto 0)
        );
    end Component;

    constant delay : time := 20 ns;
    signal A, B, Y : std_logic_vector(N-1 downto 0) := (others => '0');
    signal OP      : std_logic := '0';

begin
    -- Instantiate an instance of the ALU
    alu_inst: alu4 PORT MAP (
        A => A,
        B => B,
        OP => OP,
        Y => Y
    );

    data_proc: process is
    begin
        for i in 0 to 7 loop
            wait for delay;
            A <= std_logic_vector(unsigned(A) + 1);
        end loop;

        wait for delay;
```

```
OP <= '1';

for i in 1 to 7 loop
    A <= std_logic_vector(unsigned(A) + 1);
    for j in 0 to 3 loop
        wait for delay;
        B <= std_logic_vector((unsigned(B) + 1) mod 4);
    end loop;
end loop;

wait;

end process;
end Behavioral;
```

Appendix G Xilinx Design Constraints File

```
## Switches
set_property PACKAGE_PIN V17 [get_ports {B[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]
set_property PACKAGE_PIN V16 [get_ports {B[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]
set_property PACKAGE_PIN W16 [get_ports {B[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]
set_property PACKAGE_PIN W17 [get_ports {B[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {B[3]}]
set_property PACKAGE_PIN W15 [get_ports {A[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
set_property PACKAGE_PIN V15 [get_ports {A[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]
set_property PACKAGE_PIN W14 [get_ports {A[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]
set_property PACKAGE_PIN W13 [get_ports {A[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {A[3]}]

set_property PACKAGE_PIN R2 [get_ports {OP}]
    set_property IOSTANDARD LVCMOS33 [get_ports {OP}]

## LEDs
set_property PACKAGE_PIN U16 [get_ports {Y[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[0]}]
set_property PACKAGE_PIN E19 [get_ports {Y[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[1]}]
set_property PACKAGE_PIN U19 [get_ports {Y[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[2]}]
set_property PACKAGE_PIN V19 [get_ports {Y[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[3]}]
set_property PACKAGE_PIN W18 [get_ports {Y[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[4]}]
set_property PACKAGE_PIN U15 [get_ports {Y[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[5]}]
set_property PACKAGE_PIN U14 [get_ports {Y[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[6]}]
set_property PACKAGE_PIN V14 [get_ports {Y[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Y[7]}]

## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCC0 [current_design]
```


Appendix H Clock Wizard Tutorial

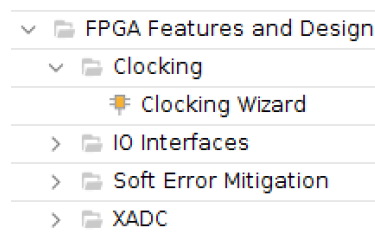
The Clock Wizard IP is a component which can be added to a design to change the speed of the clock. On the Basys3 the default clock speed is 100MHz, but by adding a clocking wizard to the design, it can be manipulated to whatever speed is necessary, within reason.

This tutorial will explain how to add a clocking wizard to a design, and how to modify the associated values.

Before beginning this tutorial, a project should be set up with the completed MIPS Pipeline, clocked at the standard 100MHz.

1. Since the source clock of the board is fixed to a specific speed, the clock needs to be scaled in order to test the MIPS Pipeline at different speeds. A **Xilinx Clocking Wizard IP Core** will be generated to manipulate the 100MHz clock to different speeds. First, the target language of the project needs to be correctly set by modifying the respective option under the Project Settings. This will ensure that Vivado generates a VHDL instantiation template. Click on **Settings** under the **Project Manager** section in the Flow Navigator and make sure that the **Target Language** is set to **VHDL**. This is likely already done, but should be checked anyway to ensure the correct template is created.

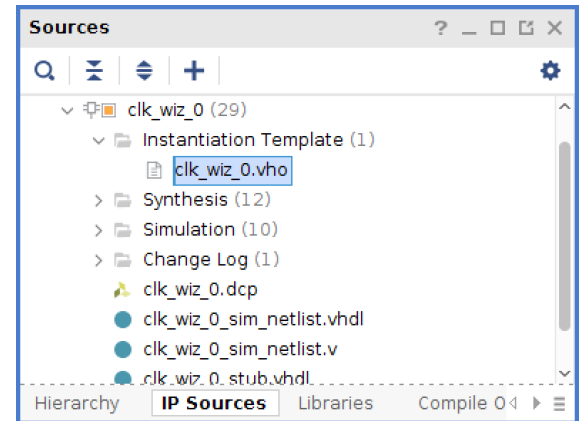
2. To add the IP core to the project, click on **IP Catalog** under the **Project Manager** section in the Flow Navigator. Navigate to **FPGA Features and Design** → **Clocking** and double click the **Clocking Wizard** to start it.



3. Leave the default options under the **Clocking Options** tag and navigate to the **Output Clocks** tab inside the wizard window. Change the clock value under the **Request Output Frequency** column from 100MHz to 125MHz. This is how the clock speed will be changed whenever that needs to occur. Deselect the **locked** option under the **Enable Optional Inputs** section at the bottom of the page. Click **OK** and then click **Generate** in the window that appears to allow Vivado to generate the necessary source files.

Board Clocking Options Output Clocks Port Renaming MMCM S				
The phase is calculated relative to the active input clock.				
Output Clock	Port Name	Output Freq (MHz)		
		Requested	Actual	
☒ clk_out1	clk_out1	25.000	25.000	
☐ clk_out2	clk_out2	100.000	N/A	

4. Now an instance of the new core needs to be instantiated and connected to the proper signals for the desired functionality. Xilinx Core generator creates a component and instance declaration template file to allow a user to easily create an instance of the new generated core. This template file can be found under the **IP Sources** tab in the **Sources** window and then expanding the navigator for new generated clock wizard core. The template file is referenced under the **Instantiation Template** folder. Open the *clk_wiz_0.vho* and copy the component block for the clocking wizard core into the architecture portion of the source. Then, copy the instantiation block and copy it under the begin portion of the architecture. Ensure the design has an input for the main FPGA 100MHz clock called “CLK_100MHZ” and connect that signal to the *clk_in1* port and create a new signal and connect it to the *clk_out1* port. This clock output signal will be the clock input signal for all clocked components in the MIPS Pipeline - allowing the components to be run at varying speeds.



5. In order to change the clock speed, return to the window where the Requested Freq of the output clock was selected and change that value appropriately. Then regenerate the IP and the change should be in place. There is no need to re-add the component or instantiation to the main architecture.

Appendix I EDAPlayground Tutorial

EDAPlayground is a web tool that is capable of simulating and synthesizing SystemVerilog, Verilog, VHDL, C++/ SystemC and other HDLs. A reference video tutorial can also be found [here](#). EDAPlayground is especially useful because it can catch and report errors that don't appear in Vivado but cause issues with simulations and results.

This tutorial will explain how to use EDA Playground to test and simulate VHDL designs.

1. Navigate to EDAPlayground.
2. Log into EDAPlayground with your account, or create a new account.
3. Add all VHDL source files by clicking on the '+' icon to the right of the default files. Copy your Testbench into the file titled *testbench.sv* in the left large pane. In the right large pane next to *design.sv*, click on the '+' icon to add all other VHDL files. You may have to copy one of your VHDL files into the design.sv file if *EDAPlayground* throws an error.
4. Choose *VHDL* on the left window pane for *Testbench + Design*.
5. Copy the test bench entity (not the file name) to the box titled *Top entity*.
6. Expand *Tools & Simulators* on the left pane, and select *Aldec Riviera Pro 2020.04*. Other simulators can also work.
7. Click the *Open EPWave after run* checkbox on the left pane.
8. Before running, save the playground to private by clicking the save button at the top of the website.
9. Click the *Run* button in the top left corner.

Appendix J ModelSim Tutorial

Modeling using data flow

Start ModelSim simulator and create a new project: File → New → Project... Name the project, set the project location (remember to avoid spaces in the names of directories and files), and click OK. A new window should appear to add files to the project. Choose Create New File. Enter the names of the files to add and click OK. Then close the “add new files” window.

Compiling files

Select the file from the project files list frame and right click on it. Select **Compile** then **Compile Selected** to just compile this file or **Compile All** for the files in the current project. If there are errors within the code or the project, a red failure message will be displayed. Double click these red errors for more detailed errors. Otherwise, if all is well then no red warning or error messages will be displayed.

Simulation with force command

To simulate, first the entity design has to be loaded into the simulator. Do this by selecting from the menu: **Simulate** → **Start Simulation**. A new window will appear listing all libraries. Unroll library **work** to see the entities (not filenames) that are in the work library (the place where all your designs will be found after successful compilation). Select the test bench for simulation and click OK. Uncheck **Enable optimization** - it is best way to maximize visibility within all parts of the design.

To see the simulation results, names of the inputs, outputs and intermediate signals must appear in the Wave window and/or List window. In the **Objects** window, Ctrl+right-click on all inputs and outputs, select **Add** → **To Wave** → **Selected signals**, and later **Add** → **To List** → **Selected signals**.

The other options are: “Signals in region” which displays all the signals listed in the signals window, and “Signals in design” displays all the signals in the project (which can be numerous when the design is dealing with other libraries. The waveform and list windows are now displayed, and the project can now be run. Run the program by typing in the ModelSim prompt:

```
run duration
```

where *duration* is a numerical value denoting time. For this full adder, run the simulation for 450 ns. The waveform and list windows now display the signal results of the run for the duration.

You can make changes to your files and restart the simulation without closing the wave window. After you make your changes, recompile your files. Then restart the simulation and run again. Your changes will be simulated.

Appendix K Common Vivado Problems

Before trying any of these, please try restarting Vivado. Unfortunately "turn it off and on again" is somewhat often the answer.

Non-Module Files

Problem

A file which shows no syntax errors appears to be a "Non-Module File".

Solution

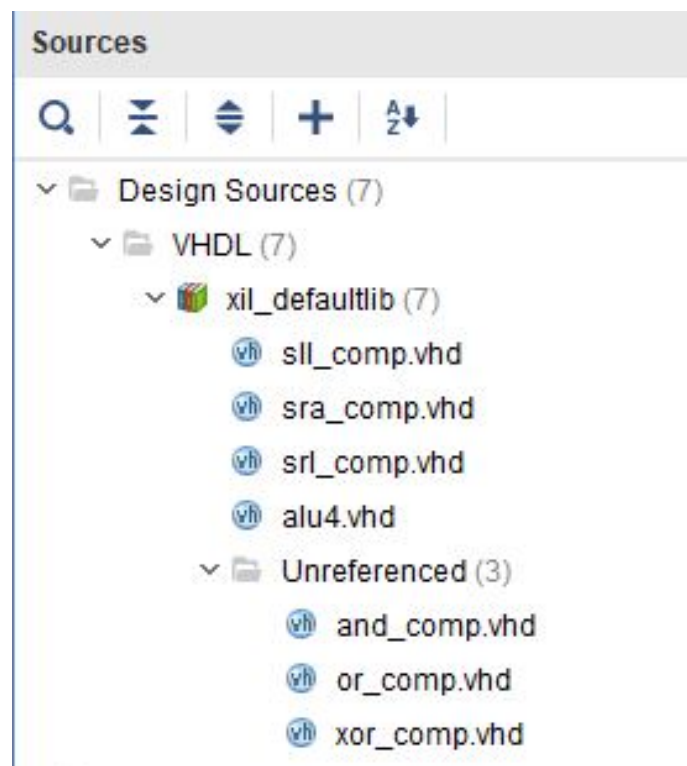
In the Tcl Console run the command: `check_syntax`. This will run a more thorough check on the files in the project and will often show errors which were not being shown earlier.

There is an option which can be added to check the syntax of a specific fileset: `check_syntax -fileset <sources_1, sim_1, constrs_1>`. This will run the same check on just the sources, simulation sources, or constraint files.

Unreferenced Files

Problem

When looking at in the Sources Window, Library Tab, some design files are listed as unreferenced (not used).



Solution

Most likely the filename and entity name of the unreferenced design file is not the same name used in the top level (the instantiating file - the one that has the port map). In short, make sure the entity name, file name and instantiation names are all the same.

Synthesis canceled

Problem

Problems that cause Xilinx synthesis to fail can often be found by changing the synthesis settings in Xilinx or by simulating in ModelSim.

Solution

To change Xilinx Settings select Settings under Project Manager in the Flow Navigator. Then select Synthesis under Project settings. Changing -flatten_hierarchy to "none" can allow Synthesis to complete. With synthesis able to complete, debugging can continue and the root cause of the problem found.

Quickly creating a project in ModelSim, compiling all the project files and simulating can generate warnings that help with debug.

A common problem ModelSim finds is undefined signals within a "to_integer" function. A wire that is declared but not driven in a memory design will often cause a failure. Xilinx synthesis will be canceled by this error but ModelSim generates a warning "numeric_std.to_integer: metavalue detected." If this warning is generated during simulation in ModelSim at any time greater than 0 ns, the signal in the "to_integer" function is undefined and probably not driven. The warning will tell you what file to look at.

Unexpected Values running stimulus, or all zeros in an input

Problem

This can happen for a number of reasons, most common is a bad testrecord array. e.g. you did type testRecord is record a,b,y : std_logic_vector(31 downto 0); op : std_logic_vector(3 downto 0); end record; but then in your declaration you did. ant tester : testArray := (

– NOT tests (x"f", x"0", x"0", x"f")

Solution

This is technically valid syntax, even though Vivado should instead throw an error. The reason this works is because vivado will simply take the next row entry, in the first row instead of making 4 bytes of f, or ["00 00 00 0f"]; it will make "1111 0000 0000 fff." The fix is to append the necessary number of 0's to each entry, e.g. [x"0000000f"].

Overutilization Error

Problem

Xilinx Vivado fails to implement the design due to an overutilization error.

Solution

If an error is encountered in the Implementation step referring to “overutilization”, it can be corrected by instructing the Synthesis step to be run in “out of context” mode. This tells Vivado to ignore the restrictions of the board when synthesizing and implementing the design. Doing so will lead to additional warnings, but it will be functional. To set this:

1. Right click on “Synthesis” in the Flow Navigator and Select “Synthesis Settings...”
2. Scroll to the bottom of the settings where there should be a text box for “More Options”
3. Type `-mode out_of_context` into the text box and resynthesize

Doing this should solve the overutilization error, but will add a number of timing and routing warnings. Since this will not be going onto a board in this form, they can be ignored for now.

Laboratory Report Guidelines

The following laboratory report guidelines apply to all CMPE-260 laboratory work, unless otherwise specified by the lab instructor.

- **Reports are due one week after the lab is performed and must be submitted at the beginning of the registered lab session.** All report submissions must be made in PDF format. All other code submissions must be made in .TXT file format. All work is to be submitted to the appropriate dropbox on MyCourses, prior to the registered lab session.
- Plagiarism in any form is not tolerated. This includes but is not limited to: copying from the lab manual, copying someone else's work, copying your own previous work, or copying from the Internet.
- Late reports will be submitted to the normal dropbox. For any late report, the student is responsible for submission and receipt: (1) informing the TA by email that the report has been submitted in the MyCourses dropbox, (2) confirming report receipt by the TA. Late penalty will be determined by the Late Policy written in the preliminary matter of this manual.
- All reports must be typed in a readable, professional font with full justification, (e.g., "justified" alignment) and 1" margins.
- Any information required for the report must be self-contained and computer generated; or else the report will be treated as not submitted. All figures should be integrated into the report, and they must be properly labeled, introduced, and explained in the text.
- Tense: Choose a tense appropriate to the content of the report, and avoid unnecessary tense shifts. For example, write about specific procedures or results from your work in past tense (since they are already completed).
- Perspective: Write the lab report only in the third person—not using "I," "me," "my," "we," "us," "our," "you," "your," etc. Avoid a writing style which talks about any person—"one," "the student," or "the instructor." For example, avoid, "One derived the truth table for a full adder," or "The student designed a parking gate control system." Such problems are easily avoided if perspective is focused solely on the technical content of what was done in lab rather than on the classroom experience.
- In the manual, at the end of each lab exercise, there is a grading sheet for collecting signatures from demonstrations in lab. This sheet must be attached at the end of the lab report. The grading sheet serves both as a grading rubric as well as a checklist to make sure all of the necessary information is included in the lab report.
- The report is composed of the following sections (in order):
 - **Cover Sheet.** The provided cover sheet template must be used.
 - **Abstract.** The abstract presents the reader with a brief but complete summary of the intent of the lab exercise. From the abstract, the reader should know what the rest of the report entails—not only what was done, how it was done, and what were the results, but also "new" concepts that were explored in the exercise. Here are two common errors in writing an abstract:

- Simply copying the objective from the lab exercise—the abstract should begin by establishing the scope of the exercise and expressing its purpose in the writer’s own words.
- Not stating the results—the abstract must be a concise but comprehensive synopsis. A consistent way to avoid this mistake is to explicitly state the success of the exercise and citing how the success was determined (e.g. successfully simulated with testbench).
- **Design Methodology.** The design methodology presents much of the theory behind the lab exercise, which was confirmed during the activity with software simulation and/or hardware experimentation. Make sure that any of the figures included not only are labeled but also are both referenced and discussed in text, (e.g., “The final design of the two-bit adder is shown in Figure 5.”). Within the report, integrate each figure next to the text that discusses it. Also, this section includes any circuit diagrams, K-maps, truth tables, etc. that were used in designing the integrated circuit. Common pitfalls include stating that “the design methodology is based on the lab manual.” The lab report should stand on its own objective technical merit without any reference to class or lab.
- **Results and Analysis.** The results and analysis section provides the reader with all results and insights obtained from completing the exercise: any graphs, tables, etc. that were collected from the lab work. Discuss how the data was obtained (e.g., using a multimeter to collect the voltage levels), the standard for comparison (e.g., truth tables), and both expected and unexpected results obtained from the lab activities. Once again, every figure, table, and schematic diagram must be properly labeled, introduced, and discussed in text. Here are some common pitfalls:
 - Putting conclusions here.
 - Not comparing results obtained with the corresponding aspect(s) in Design Methodology.
- **Conclusions.** The conclusion section states whether the exercise as a whole was properly executed and completed. It discusses any facilitators and hurdles encountered in performing the exercise. In addition, it describes how theories and techniques (learned in class) were applied in the lab exercise, the lessons learned from the lab activities, and what could have been done, if anything, to make the exercise go smoother.
- **Questions.** For each question in the manual at the end of the lab exercise, the questions section copies the question and gives an answer. Some of the questions can be challenging and require leveraging concepts learned from the lecture, the lab activities, and sometimes additional reference material. Students are encouraged to attempt answering the questions before the lab session ends, and may seek for help from the lab instructor and the TA. Attempting to solve a question is better than leaving it blank—even if the answer is wrong.
- Writing composition points will be awarded if the lab report is written with enough concise, self-contained information that someone with similar background could read only the lab report and be able to perform the same exercise to obtain similar results. For example, your lab report should contain enough detail so that you could reread it three years from now to know exactly how to repeat the exercise and get very similar results.
- Further details and tips on how to write a report can be found in the sample report that been posted on MyCourses. Failure to follow guidelines in the sample report or in this section will result in point deductions.

- In the event that not all sign-offs for in-lab demonstration are obtained within the on-time submission window, a student may choose to write a report on the theory and goal of the exercise and state that the exercise was unsuccessful. In this case, the Design Methodology should remain as if the exercise was completed, but the Results and Analysis should focus on the expected results in comparison to the actual results. The student should then emphasize on what had gone wrong and how things could potentially be fixed in the future. By doing this, it is possible to earn points for the report without losing extra points for not having the exercise and report completed.